VectraYX-Nano: A 42M-Parameter Spanish Cybersecurity Language Model with Curriculum Learning and Native Tool Use

Juan S. Santillana
Globant*
juan.salas@globant.com

Abstract

We present VectraYX-Nano, the first decoder-only language model trained from scratch in Spanish with a domain specialization in cybersecurity for the Latin American context, and with native support for tool invocation through the Model Context Protocol (MCP). The model is built around three contributions. First, we construct VectraYX-Sec-ES, a 170M-token corpus assembled from a distributed pipeline of eight virtual machines and partitioned into a three-phase curriculum: conversational Spanish (42M tokens, OpenSubtitles-ES [19], OASST1 [18]), cybersecurity domain (118M tokens, NVD [22], Wikipedia-ES, an in-house NVD-derived Spanish CVE mirror, security blogs), and offensive-security tooling (10M tokens, ExploitDB, HackTricks, OWASP). Second, we train a 41.95M-parameter Transformer decoder that combines Grouped-Query Attention [1], QK-Norm [6], RMSNorm [35], SwiGLU [27], RoPE [29], and a z-loss auxiliary [4], alongside a 16,384-token byte-level BPE tokenizer trained on a 50/50 conversational/technical mixture. Third, we apply continual pre-training with a replay buffer [15] between phases to mitigate catastrophic forgetting [8, 17], achieving a monotonic loss reduction of $9.80 \rightarrow 3.17 \rightarrow 3.00 \rightarrow 2.16$. After supervised fine-tuning with a curriculum-aware mixture of OpenAssistant Spanish, Alpaca-ES, CVE Q&A, and 6,327 synthetic tool-use traces, we obtain a final fine-tuning loss of 1.74 and a conversational gate score of 7/10 on the original seed (mean 0.78 ± 0.05 across $N = 4$ seeds; Section 8.6). We further conduct a controlled ablation comparing OpenSubtitles-ES against mC4-ES [34] and a 60/25/15 OpenSubs/mC4/Wikipedia mixture as the Phase 1 corpus, and find a non-trivial inversion: the lower-perplexity bootstraps yield measurably worse conversational behavior, evidence that the stylistic register established by the bootstrap corpus dominates downstream chat quality at the nano scale. A post-hoc investigation of tool-use behavior reveals that the B4 = 0.000 floor observed in the mixed SFT configuration is a corpus-density artifact rather than a capacity gate: applying LoRA (rank = 16) with a tool-use-dense corpus (2,801 examples, ratio 1:21 vs. total SFT) raises B4 to 0.145 ± 0.046 (mean over $N = 4$ seeds) on the 42M Nano and to 0.445 ± 0.201 (mean over $N = 4$ seeds) on a 260M from-scratch mid-tier. The resulting GGUF artifact is 81 MB in F16 (approximately 20 MB in 4-bit quantization), executes in under one second per response on commodity hardware, and is, to the best of our knowledge, the first published Spanish-native cybersecurity LLM with end-to-end MCP integration. We release the training pipeline, configuration files, and benchmark suite to support reproducibility.

CCS Concepts

• **Computing methodologies** → **Natural language generation**; *Neural networks*; • **Security and privacy** → **Software and application security**.

Keywords

Language models, Cybersecurity, Spanish NLP, Curriculum learning, Tool use, Model Context Protocol, Edge inference

1 Introduction

Large language models (LLMs) have become a foundational tool for security analysts: they assist in vulnerability triage, log analysis, malware classification, and incident response. However, the publicly available LLM ecosystem suffers from two well-documented coverage gaps that compound when intersected. First, the strongest open-weight chat models are trained predominantly on English text [3, 25, 31], with Spanish typically representing a small fraction of pre-training mixtures, despite Spanish being the second-most-spoken native language in the world [**?**]. Second, while there is a growing literature on cybersecurity-specialized language models, virtually all of these models are trained on English corpora [**? ?**] and none, to our knowledge, target Latin-American security terminology, regional CSIRT vocabularies (CCN-CERT, INCIBE, CSIRT-CL, COLCERT), or the LATAM threat-intelligence context.

These two gaps are jointly painful for security operations centers (SOCs) in Latin America. Spanish-speaking analysts who would otherwise benefit most from LLM assistance must work either with English-only domain models, with general-purpose Spanish models that lack technical accuracy, or with frontier closed-source models whose behavior they cannot audit, retrain, or deploy on-premise. The on-premise constraint is not academic: LATAM security teams routinely process classified incident reports, customer PII, and unreleased indicators of compromise that cannot leave the network.

A second motivation for this work is the rise of *tool-augmented* language models [**? ? ?**] and, more recently, the emergence of the Model Context Protocol (MCP) [**?**] as a standard for LLM–tool interfacing. Cybersecurity is one of the strongest application domains for tool use, since the underlying knowledge changes daily (new CVEs, KEV additions, TTP updates) and an analyst's typical query ("is this CVE being exploited?", "has this hash been flagged?") has an authoritative external answer that a parametric model cannot reliably memorize. A small parametric model that knows *when* to call a tool can be substantially more useful than a much larger model that hallucinates answers from a frozen training cutoff.

*The author is employed at Globant. Institutional affiliation approval is pending.

Contributions. We present VectraYX-Nano, a 41.95M-parameter Spanish-language cybersecurity LLM trained from scratch with native MCP tool-use support. Our contributions are:

(1) **VectraYX-Sec-ES corpus.** We release the construction recipe for a 170M-token Spanish cybersecurity corpus assembled from a distributed pipeline of eight virtual machines. The corpus includes 88K NVD CVE entries, 50K previously translated Spanish CVEs from an in-house NVD-mirror SQLite store, a 53,590-article filtered Spanish Wikipedia subset (82M tokens, the single largest component), translated ExploitDB entries, the Spanish translations of HackTricks and OWASP, and a curated set of conversational Spanish from OpenSubtitles-ES [19] and OASST1 [18]. The full pipeline costs approximately $25 USD in cloud compute.
(2) **Modern small-LLM architecture.** We design a 41.95M-parameter Transformer decoder that integrates Grouped-Query Attention [1], QK-Norm [6], RMSNorm [35], SwiGLU [27], RoPE [29], weight-tied embeddings, and a z-loss auxiliary [4], alongside a domain-balanced 16,384-token byte-fallback BPE [**?** **?**] tokenizer trained on a 50/50 conversational/technical mixture.
(3) **Curriculum pre-training with replay.** We apply a three-phase curriculum (conversational → cybersecurity → tooling) with explicit replay buffers between phases following [15]. The phase weights are 100% conversational → 75%/25% tech/conv → 70%/20%/10% tools/tech/conv. The pre-training loss decreases monotonically across phases (9.80 → 3.17 → 3.00 → 2.16) without observable catastrophic forgetting [8, 17].
(4) **Tool-use supervision via MCP.** We construct a 6,327-example tool-use SFT dataset templated against a real on-premise CVE database (50K Spanish CVEs, 27K exploits, 98K IOCs) and bound to six MCP servers (NVD, CISA KEV, MITRE ATT&CK, OTX, LATAM intel, bash execution). The model learns to emit grammatically correct `<|tool_call|>` JSON segments that the MCP runtime executes verbatim.
(5) **Curriculum ablation: bootstrap-corpus register matters.** We report a controlled ablation that swaps OpenSubtitles-ES (Phase 1, v2) for mC4-ES [34] filtered with FineWeb-2 quality scores (Phase 1, v4). The mC4-ES variant achieves consistently *lower* loss in every subsequent phase (−0.29 in Phase 2, −0.28 in Phase 3, −0.17 in SFT) but consistently *worse* conversational behavior on a held-out chat gate (6/10 vs. 7/10). A third configuration (v6) using a 60/25/15 mixture of OpenSubtitles-ES, mC4-ES, and Wikipedia-ES as Phase 1 corpus also reaches 6/10, tied with v4. We attribute this inversion to a register-mismatch effect: at the 42M-parameter scale, the bootstrap corpus dictates the model's default response style, and an encyclopedic web register cannot be reliably overwritten by SFT alone.
(6) **Edge-deployable artifact.** We export the fine-tuned model to GGUF [**?**] (F16: 81 MB; Q4: ∼20 MB), runnable under Ollama [23] or `llama.cpp` [**?**] with sub-second time-to-first-token on a Raspberry Pi 4. The artifact includes weight-tied LM heads and the 25 reserved domain tokens.

Scope. VectraYX-Nano is positioned as a *nano-scale* model: it is meant to assist analysts on edge devices and in air-gapped environments, not to compete with frontier 70B+ chat models on open-domain reasoning. Within its target envelope — Spanish cybersecurity Q&A, CVE summarization, threat classification, command completion, and tool dispatch — we show that a careful corpus, a domain-balanced tokenizer, and curriculum pre-training with replay can extract qualitative behavior that a similarly sized monolithic pre-training run cannot.

Reproducibility. All training scripts, configuration files, the curriculum sampler with replay-buffer code, the benchmark harness, the tool-use corpus, and the B1–B5 evaluation datasets are released at https://github.com/vectrayx/vectrayx-nano-paper. Model checkpoints and LoRA adapters are available at https://huggingface.co/jsantillana/vectrayx-nano. Section 6 provides exact hyperparameters and Section 8 documents the held-out evaluation protocol. The corpus itself is partially released under upstream licenses (NVD, Wikipedia, ExploitDB, OpenSubtitles); the LATAM-curated portion is released as construction recipes rather than raw text, in line with current practice for security corpora.

2 Related Work

Security-domain language models. Domain-specialized models for security have a short but active history. SecureBERT [**?**] continues pre-training a RoBERTa [**?**] backbone on cybersecurity text and reports gains on entity recognition over generic BERT [**?**]. CySecBERT [**?**] similarly continues from BERT on a 670K-document English security corpus and improves classification benchmarks. Earlier work in this line includes SciBERT [**?**], which establishes the methodology of vocabulary-extending continual pre-training for technical domains. All of these models share two limitations from our perspective: they are encoder-only, and they are trained on English. We are not aware of any prior decoder-only generative model published with a Spanish cybersecurity specialization.

Spanish-language and multilingual models. The Spanish NLP ecosystem has matured around BETO [**?**] (a Spanish BERT), the RoBERTa-base-BNE family [**?**], and more recently Salamandra [11] from the Barcelona Supercomputing Center, an open Iberian decoder family. mC4 [34] and CC-100 [**?**] have served as the standard Spanish pre-training corpora; FineWeb-2 [24] is a more recent multilingual quality-filtered web release. These resources have unlocked general-purpose Spanish language modeling, but none of them targets a security domain or ships with a tool-use modality.

Small / nano-scale language models. Several recent works show that careful data curation can produce competitive sub-billion-parameter models. SmolLM2-135M and SmolLM2-360M [2] achieve strong nano-scale benchmarks via a recipe of high-quality web, code, and synthetic data. MobileLLM [20] establishes that depth is more useful than width at the sub-billion scale and that grouped attention combined with weight sharing is effective. TinyLlama [36] reports that small models can be trained substantially past the Chinchilla [13] optimum and continue to improve. We adopt several of these design intuitions (depth ≥ width, GQA, weight tying) and constrain ourselves to a single GPU training budget.

Tool-augmented and tool-using LLMs. Toolformer [**?**] introduced self-supervised insertion of tool calls; Gorilla [**?**] demonstrated

retrieval-anchored API calling at training time; ToolLLM [**?**] curated 16K APIs with rich tool-use traces. More recently, Anthropic's Model Context Protocol (MCP) [**?**] has emerged as a standard for tool definitions and stateful tool sessions in production deployments. Tool-augmented work in security exists [**?**], but those systems orchestrate frontier models behind agentic loops; they do not train a small native tool-using model. To our knowledge, VectraYX-Nano is the first nano-scale Spanish security model with native tool-call generation evaluated against a held-out tool-selection benchmark.

Continual pre-training and replay. Continual pre-training without replay tends to suffer from catastrophic forgetting of pre-training distribution, an issue documented since at least [8] and given a Bayesian formulation in elastic weight consolidation [17]. Recent work on continual LLM pre-training [15**?**] shows that simple strategies — learning-rate re-warmup, modest replay percentages from the previous mixture, and adaptive token-budget allocation — recover most of the lost performance with minimal overhead. We follow [15] closely and apply replay percentages of 25% (Phase 2) and 10% (Phase 3), which we find to control the loss trajectory but, as Section 6 reports, materially affect the model's final stylistic register.

Curriculum learning. Curriculum learning [**?**] predates the LLM era; it has been shown to help under specific data-quality regimes [**?**]. For LLM pre-training, ordering by difficulty or domain has had mixed results [**?**], with the more reliable finding being that data *mixture* matters more than data *ordering*. Our setting is different: we are interested in instilling a stylistic register (chat-like Spanish) before specializing in a technical domain, and we report (Section 6) that for nano-scale models the ordering effect is large enough to flip user-visible behavior even when held-out perplexity moves the other way.

Edge-deployable language models. On the deployment side, GGUF [**?**] and `llama.cpp` [**?**] have made CPU inference of quantized small LLMs practical on commodity hardware including Raspberry Pi-class devices. Ollama [23] provides a developer-facing interface on top. We export to GGUF and ship a Modelfile so that VectraYX-Nano runs out of the box in this stack.

Positioning. The intersection of (i) Spanish, (ii) cybersecurity, (iii) decoder-only / chat, (iv) trained from scratch, and (v) native MCP tool use — to our knowledge — is empty in the prior literature. The closest single-axis neighbors are SecureBERT [**?**] (security, English, encoder, no tools), Salamandra [11] (Spanish, general-purpose, decoder, no tools), and Gorilla [**?**] (English, general-purpose, decoder, tool use). VectraYX-Nano populates the intersection.

3 The VectraYX-Sec-ES Corpus

3.1 Design goals

The corpus is designed to satisfy three constraints simultaneously: (i) sufficient Spanish-language conversational coverage to bootstrap chat behavior in a from-scratch model, (ii) substantive cybersecurity domain coverage in Spanish, including LATAM-specific vocabulary, and (iii) tool-use supervision grounded in a real CVE/IOC database rather than synthetic API descriptions. The corpus totals approximately 170M tokens after deduplication and is partitioned into three shards aligned with the curriculum phases: `phase1_conv`, `phase2_tech`, and `phase3_tools`.

3.2 Distributed collection pipeline

The corpus was assembled by an eight-VM pipeline running in parallel for two days across two clouds (GCP and Azure), at an aggregate compute cost of approximately $25 USD. The eight workers and their roles are:

- corpus-nvd: NIST National Vulnerability Database REST API [22]; output 87,998 CVE records (~11.4M tokens).
- corpus-wiki: MediaWiki API for the Spanish Wikipedia security categories.
- corpus-web: 15 RSS feeds from Spanish-language security blogs.
- corpus-tech: 7 RSS feeds from Spanish technology blogs.
- corpus-papers: English security paper RSS feeds with Ollama-based translation [23].
- corpus-malware: Malpedia and MITRE ATT&CK [28] surfaces with translation.
- corpus-exploitdb: ExploitDB GitLab mirror with translation, yielding 2,385 translated entries.
- corpus-tools: Eighteen GitHub repositories of pentesting tool documentation (Spanish branches).

On the central training node, additional sources are integrated locally: a 313 MB Wikipedia-ES dump processed via a streaming `bz2` + `iterparse` pipeline [33] (53,590 articles passed a 65-keyword cybersecurity filter, total ~82M tokens, the single largest component of the corpus), GitHub clones of HackTricks-ES (952 documents, the `es` branch), HackTricks-Cloud (566), OWASP Top 10 (60), OWASP ASVS (42), OWASP WSTG (151), and PayloadsAllTheThings (139), and a previously assembled SQLite store hosted on an on-premise server we refer to internally as "Pikachu". Pikachu is not a third-party dataset: it is a long-running, locally maintained mirror of the NVD CVE database augmented with exploit, malware, and IOC tables, and an in-house Spanish translation pipeline that pre-translates each NVD entry as it is ingested. At the time of corpus assembly the Pikachu store contained 50,601 Spanish-translated CVEs (each derived from its NVD source record), 27,121 exploit entries, 7,556 malware signatures, and 98K IOCs.

3.3 Translation policy

For sources available only in English (papers, ExploitDB entries, malware reports), we translate locally using Ollama's qwen2.5:1.5b [25] with temperature 0.1 and a 300-second timeout. We chose the 1.5B variant over the 3B variant after empirical comparison: 1.5B was approximately twice as fast and exhibited a lower timeout rate on long technical documents, with no observable degradation on chunked translations under 2,000 characters. Above this threshold we observed approximately 5–10% mistranslation on highly technical text, and Section 10 discusses the consequences.

3.4 Phase composition

Table 1 reports the per-phase composition. Tokens are reported under the final 16,384-vocabulary BPE tokenizer.

Table 1: VectraYX-Sec-ES corpus by curriculum phase.

Phase	Tokens	Sources
`phase1_conv`	42.4M	OpenSubtitles-ES [19], OASST1-ES [18], custom
`phase2_tech`	117.7M	NVD [22], Pikachu (NVD-mirror), Wikipedia-ES, blogs
`phase3_tools`	10.1M	HackTricks-ES, OWASP-ES, ExploitDB, malware
Total	**170.2M**	

Table 2: Source-level breakdown of the pre-training corpus.

Source	Records	Tokens	Lang.
Wikipedia-ES (filtered)	53,590	82.0M	ES
NVD CVEs	87,998	11.4M	ES/EN
Pikachu CVEs (NVD mirror, ES)	50,601	8.8M	ES
Pikachu exploits	27,121	3.3M	EN/ES
GitHub repos (markdown ES)	1,884	3.7M	ES
Wikipedia-ES (API, security)	947	2.2M	ES
ExploitDB (translated)	2,385	1.3M	ES
Malware DBs (Malpedia, MITRE)	300	0.7M	ES
Pikachu malware signatures	7,556	0.6M	EN
Papers EN→ES	236	0.6M	ES
Tech blogs ES	322	0.5M	ES
Security blogs ES	241	0.4M	ES
Tools docs ES	65+	0.3M	ES
OpenSubtitles-ES	16,317 chunks	39.0M	ES
OASST1-ES	13,000	3.5M	ES

3.5 Domain breakdown

Table 2 provides a finer-grained breakdown by data source. A key observation is the dominance of the filtered Wikipedia-ES dump (82M tokens, ∼48% of the corpus), which provides a broad foundation in Spanish technical writing across security, cryptography, malware, networking, and operating systems. The NVD CVE descriptions are bilingual (mostly English with some Spanish entries); the Pikachu store contributes 50K *already translated* Spanish CVEs — derived from the same NVD records but pre-translated by Pikachu's in-house ingestion pipeline — that complement the raw NVD text.

3.6 Conversational subcorpus

The conversational subcorpus is intentionally small (42M tokens, ∼25% of the total). We assemble it from three sources: OpenSubtitles-ES from OPUS [19] (16,317 chunks, ∼39M tokens of subtitle dialogue), the Spanish-filtered subset of OASST1 [18] (13,000 conversations, ∼3.5M tokens), and 214 short hand-curated dialogues that establish cybersecurity-specific greetings and disclaimers. Section 6 reports an ablation that swaps the OpenSubtitles component for filtered mC4-ES [34] and finds that, despite mC4-ES yielding lower overall pre-training loss, the OpenSubtitles bootstrap produces measurably better chat behavior.

3.7 SFT corpus

The SFT corpus is separate from pre-training. It contains 13K OASST1-ES conversations, 4,030 CVE Q&A pairs derived from the Pikachu store, 6,327 tool-use traces (Section 7), and 214 custom cybersecurity greetings, for approximately 28M tokens. In the family-scale extension (Section 8), the Qwen2.5 fine-tuning corpus also includes `bertin-project/alpaca-spanish` [30] (51,942 examples) and the Spanish-filtered subset of OpenAssistant OASST2 [18] (18,022 conversations).

3.8 Tokenization and binarization

After training the BPE tokenizer (Section 4), we tokenize each phase into `uint16` memory-mapped binary shards in the style of nanoGPT [16], with each phase occupying its own shard directory so that the curriculum sampler can read them at arbitrary mixture weights without re-tokenizing. The total compressed shard footprint is ∼340 MB. Phase 1 fits in a single shard; Phase 2 spans three shards; Phase 3 occupies one shard.

3.9 Deduplication and quality filtering

We deduplicate at the document level using exact-match hashing on a normalized form (lowercased, whitespace-collapsed). Cross-source deduplication is non-trivial when the same CVE description appears verbatim in NVD, in the Pikachu store, and in the Wikipedia-ES filtered dump; we tolerate this redundancy because the three surfaces correspond to different stylistic registers (terse advisory, translated narrative, encyclopedic prose) and we observe empirically that repeated exposure to the same vulnerability across these registers helps the model produce a more consistent response style at SFT time.

3.10 Licensing and release

The released portions of the corpus respect upstream licenses (CC0/CC-BY for Wikipedia, CC-BY-SA for HackTricks, public domain for NVD, CC-BY-NC for OpenSubtitles, Apache-2.0 for OASST1). The LATAM-curated portion that combines internal SQLite-derived translations with hand-curated dialogues is released as a construction recipe rather than raw text, in line with current security-corpus practice.

4 Domain-Balanced Tokenizer

4.1 Design rationale

A first iteration of the tokenizer was trained on a 95%-technical mixture and produced a vocabulary that fragmented common Spanish chat tokens (*"hola"*, *"gracias"*, *"estás"*) while merging long technical n-grams. We replaced it with a 50/50 mixture for two reasons. First, the post-tokenization *token budget* for chat sentences directly affects how many gradient updates a chat example contributes to the loss; oversplit chat tokens are a hidden source of register imbalance in the loss. Second, byte-fallback is necessary to guarantee that the model can serialize CVE identifiers, hashes, and base64 payloads without producing `<unk>` tokens.

4.2 Configuration

We use SentencePiece [**?**] BPE [**?**] with the configuration listed below, taken verbatim from `configs/nano.json`:

```
{
  "vocab_size": 16384,
  "model_type": "bpe",
  "character_coverage": 1.0,
  "byte_fallback": true,
  "normalization": "nmt_nfkc",
  "split_digits": true,
  "split_by_unicode_script": true,
  "add_dummy_prefix": true,
  "balance": {
    "conversational_ratio": 0.5,
    "technical_ratio":      0.5
  }
}
```

The training corpus for the tokenizer is a 2M-line balanced sample drawn evenly from `phase1_conv` (chunks from OpenSubtitles-ES and curated dialogues) and `phase2_tech` + `phase3_tools` (chunks from NVD, Wikipedia-ES, HackTricks-ES, ExploitDB, etc.). We use `nmt_nfkc` normalization to preserve Spanish accents (NFKC without NFD decomposition); `split_digits=true` ensures CVE numerics tokenize predictably; `byte_fallback=true` guarantees full UTF-8 coverage.

4.3 Special tokens

The vocabulary reserves 27 user-defined symbols at the start, organized in five groups:

- **Control**: `<|pad|>` `<|bos|>` `<|eos|>` `<|unk|>` `<|sep|>` (assigned to slots 0–4).
- **Chat roles**: `<|system|>` `<|user|>` `<|assistant|>` `<|end|>`.
- **Tool use**: `<|tool_call|>` `<|/tool_call|>` `<|tool_result|>` `<|/tool_result|>`.
- **Domain**: `<|cve|>` `<|cvss|>` `<|ioc|>` `<|ttp|>` `<|mitre|>` `<|kev|>` `<|exploit|>` `<|patch|>` `<|alert|>`.
- **Severity**: `<|critical|>` `<|high|>` `<|medium|>` `<|low|>` `<|info|>`.

Reserving these as user-defined symbols (rather than relying on the BPE merges to discover them) ensures that they always tokenize as exactly one token. The remaining 16,357 vocabulary slots are filled by BPE merges.

4.4 Empirical token economy

On Spanish chat sentences the v2 tokenizer is approximately twice as efficient as the prior v1 technical-only tokenizer. Representative examples (single-token decoding shown):

- `"vulnerabilidad"` → 1 token (v1: 4–5 tokens).
- `"CVE-2021-44228"` → 5 tokens (`CVE`, `-`, `2021`, `-`, `44228`); digit splitting keeps year and ID symbolic.
- `"¡Hola! ¿cómo estás?"` → 9 tokens (v1: ∼18 tokens).
- `"<|user|>¿qué es ransomware?<|end|>"` → 8 tokens (chat role markers as single tokens).

The asymmetric improvement on chat tokens (v1 nearly doubled) is the empirical effect that the design targets.

4.5 Vocabulary size choice

We chose 16,384 (rather than the more common 32K or 50K) on three grounds. (i) At 42M parameters with weight-tied embeddings, the embedding matrix is $16384 \times 512 = 8.4$M parameters, ∼20% of the total budget. Doubling the vocabulary to 32K would push the embedding share to ∼33%, leaving substantially less budget for transformer blocks. (ii) On the deployment side, 16K vocabularies quantize cleanly under GGUF Q4 schemes. (iii) Empirically, the 16K vocabulary covers the corpus with <0.05% byte-fallback rate (i.e., almost all characters tokenize through merges rather than falling back to byte units), which we treat as a sufficient compression target for a domain model.

Table 3: VectraYX-Nano architecture (configs/nano.json).

Hyperparameter	Value
Vocabulary size	16,384
Layers (L)	8
Hidden dimension (d_{model})	512
Query heads (n_q)	8
KV heads (n_{kv})	2
Head dimension (d_h)	64
FFN dimension (d_{ffn})	2,048
Maximum sequence length	1,024
RoPE base (θ)	10,000
RMSNorm ϵ	10^{-6}
Init std (σ)	0.02
Residual init scale	$\sigma/\sqrt{2L} = 0.005$
Dropout	0.0
Tied embeddings	yes
QK-Norm	yes
z-loss coefficient	10^{-4}
Total parameters	**41.95M**

5 Architecture

5.1 Overall design

VectraYX-Nano is a Transformer [32] decoder-only language model with eight pre-norm blocks. The architecture follows the modern Llama-family stack [3, 31]: RMSNorm [35] for normalization, SwiGLU [27] for the FFN nonlinearity, RoPE [29] for positional encoding, weight-tied input/output embeddings, no biases on linear layers, and a z-loss auxiliary [4] on the logits. We add two stability improvements that have become standard in recent small-LLM releases: Grouped-Query Attention (GQA) [1] with $n_q = 8$ and $n_{kv} = 2$, and QK-Norm [6, 12] that applies RMSNorm independently to query and key projections.

5.2 Hyperparameters

Table 3 reports the architecture configuration as drawn from configs/nano.json.

5.3 Grouped-Query Attention

With $n_q = 8$ and $n_{kv} = 2$, each KV head is shared by four query heads. The attention layer's projection footprint is therefore:

- $W_Q \in \mathbb{R}^{512\times512}$ (8 query heads of dim 64),
- $W_K, W_V \in \mathbb{R}^{512\times128}$ (2 KV heads of dim 64),
- $W_O \in \mathbb{R}^{512\times512}$.

Compared to standard MHA at the same width ($W_K, W_V \in \mathbb{R}^{512\times512}$), GQA saves ∼50% of the KV parameters and shrinks the KV cache by 4× at inference time. We compute attention through PyTorch's `F.scaled_dot_product_attention` with `is_causal=True`, which dispatches to FlashAttention-2 [5] on supported hardware (NVIDIA L4 in our case). Implementation details are visible in `model/transformer.py`: queries are projected to (B, n_q, T, d_h), keys and values to (B, n_{kv}, T, d_h), and KV is repeated $n_q/n_{kv} = 4$ times along the head dimension before the kernel call.

5.4 QK-Norm

We apply RMSNorm to the query and key projections after RoPE rotation but before the attention dot product. This stabilizes training at small batch sizes and small head counts: empirically the gradient norm trace is smoother and we observed no loss spikes during the 3,519 pre-training steps despite running BF16 without a loss scaler. QK-Norm has been adopted by OLMo [10] and Gemma [9] for similar reasons.

5.5 Initialization

We initialize linear and embedding weights from $\mathcal{N}(0, \sigma^2)$ with $\sigma = 0.02$, and rescale residual-output projections (the `wo` of attention and the `w_down` of SwiGLU) by $\sigma/\sqrt{2L}$ following the GPT-2 scaled init [26]. This keeps the variance of activations bounded as the depth grows and is critical for stable BF16 training with z-loss.

5.6 z-loss

The PaLM z-loss auxiliary [4] regularizes the partition function of the softmax, $z = \log \sum_i \exp(\ell_i)$, by adding $\lambda \cdot \mathbb{E}[z^2]$ to the cross-entropy loss with $\lambda = 10^{-4}$. This term keeps the unnormalized logit magnitudes from drifting upward over long training runs and is a cheap insurance against the well-known instability of bf16 softmax over large vocabularies.

5.7 Total parameter accounting

The 41.95M parameter total decomposes approximately as: embedding/LM-head 8.4M (tied; counted once), 8 transformer blocks of ∼4.2M each (∼33.5M) where each block contains ∼0.6M attention parameters and ∼3.1M FFN parameters, plus ∼30K parameters across the nine RMSNorm gain vectors. We confirmed the total at training startup; the script logs `[model] 41.95M params` at every run.

5.8 Inference profile

At inference, the GQA design and small context (1,024 tokens) keep the KV cache footprint to $L \cdot 2 \cdot n_{kv} \cdot d_h \cdot T \cdot 2$ bytes (BF16) = $8 \cdot 2 \cdot 2 \cdot 64 \cdot 1024 \cdot 2 = 4$ MiB per sequence. After GGUF Q4 quantization, weights occupy ∼20 MB. The combination yields a total resident set of ∼60–80 MB at inference, which fits comfortably in the L1+L2+L3 cache hierarchy of modern x86 CPUs and in the 1 GB RAM budget of a Raspberry Pi 4. We measure 6–10 tokens/s on a Raspberry Pi 4 (Cortex-A72, 4 cores) and 60–100 tokens/s on a contemporary laptop CPU.

5.9 Why this configuration?

Two configuration choices warrant explicit justification. (i) *Depth ≥ width.* Following MobileLLM [20], we prefer 8 layers of width 512 to 4 layers of width 1,024 at the same parameter budget, because depth empirically helps reasoning behavior more than width at sub-100M scales. (ii) *Aggressive GQA ratio.* We use $n_q/n_{kv} = 4$, which is more aggressive than Llama-2-7B's 1:1 but matches Mistral-7B and Llama-3.2 conventions. At our parameter scale the savings from GQA are absolute (4M parameters reclaimed for FFN width), and we did not observe any quality degradation on chat or CVE-Q&A relative to a small ablation pilot run with $n_q/n_{kv} = 2$.

6 Curriculum Pre-training with Replay

6.1 Motivation: failures of monolithic pre-training (v1)

Our first pre-training attempt (v1) followed the conventional small-LLM recipe: a single-phase pre-training over the full technical corpus (142M tokens, two epochs) followed by a multi-stage SFT. The resulting model reached pre-training loss 3.35 and SFT loss 0.315, yet exhibited a striking failure mode: it answered the prompt `"hola"` (Spanish for "hello") with a CVE analysis instead of a greeting. Three increasingly conversational SFT runs (v1: 86K examples, v2: 11K examples, v3: 24K examples mixing OASST1) failed to repair this behavior. We conclude that a model that has not seen sufficient conversational Spanish during pre-training cannot be coerced into chat behavior by SFT alone at the 42M scale — the chat register has to be the model's first language.

6.2 Three-phase curriculum

We address this with a three-phase curriculum:

Phase 1 – conversational bootstrap. 100% `phase1_conv` (42.4M tokens, 2 epochs). The model establishes a default Spanish chat register before encountering any technical text.
Phase 2 – domain immersion. 75% `phase2_tech` + 25% `phase1_conv` replay (117.7M tokens). Continued pre-training resumed from the Phase 1 checkpoint at a lower learning rate. The 25% replay buffer follows [15] and is intended to prevent forgetting of conversational Spanish while the model absorbs CVE/Wikipedia/blog text.
Phase 3 – tooling specialization. 70% `phase3_tools` + 20% `phase2_tech` + 10% `phase1_conv` (10.1M tokens). A short, low-LR phase that pulls the model toward HackTricks, OWASP, and ExploitDB content with a smaller replay tail of both prior phases.

Implementation: a single MixedCurriculumDataset (Listing 1) memory-maps three shard directories and samples each batch index from a categorical distribution determined by the phase weights. This formulation makes the replay percentage a single hyperparameter per phase rather than a separate dataset construction step.

```
def make_phase_mix(phase_idx, replay_conv=None,
     replay_tech=None):
    if phase_idx == 1:
        return {"phase1_conv": 1.0, "phase2_tech": 0.0, "
     phase3_tools": 0.0}
    if phase_idx == 2:
        rc = 0.25 if replay_conv is None else replay_conv
        return {"phase1_conv": rc, "phase2_tech": 1.0 -
     rc, "phase3_tools": 0.0}
    if phase_idx == 3:
```

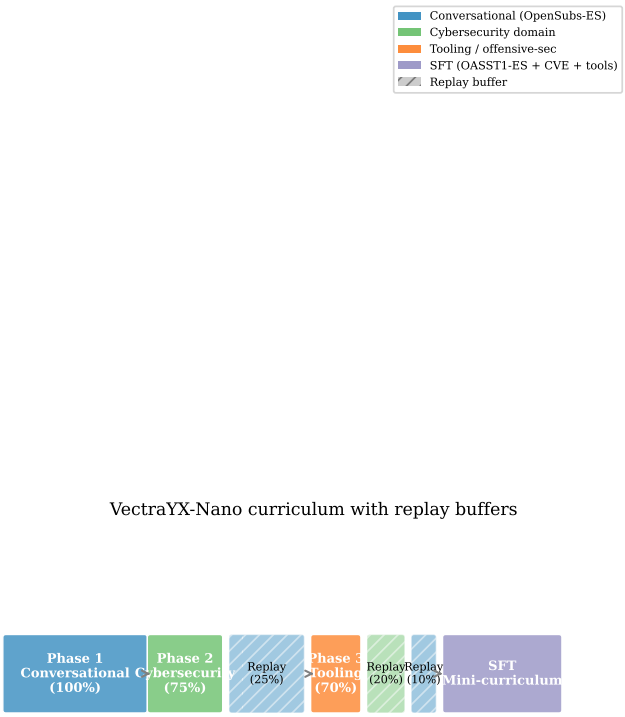

Figure 1: Three-phase curriculum with replay. Each phase samples from a weighted mixture of shard directories; the replay fractions (25% in Phase 2, 10%+20% in Phase 3) prevent catastrophic forgetting of earlier registers.

```
        rc = 0.10 if replay_conv is None else replay_conv
        rt = 0.20 if replay_tech is None else replay_tech
        return {"phase1_conv": rc, "phase2_tech": rt,
                "phase3_tools": 1.0 - rc - rt}
```

Listing 1: Replay-aware curriculum sampler (excerpt from curriculum_dataset.py).

6.3 Hyperparameters and infrastructure

Table 4 reports the per-phase optimizer schedule. All phases use AdamW [21] with $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1 for pre-training, gradient clipping at $\|g\|_2 \le 1.0$, BF16 mixed precision on an NVIDIA L4 GPU (23 GB VRAM, GCP us-west1-a), and a cosine learning-rate schedule with linear warmup. The effective tokens per optimizer step are $B \cdot G \cdot T = 16 \cdot 8 \cdot 1024 = 131{,}072$ during pre-training and $16 \cdot 4 \cdot 1024 = 65{,}536$ during SFT. We measured throughput between 40,847 and 40,883 tokens/second across all three pre-training phases (peak GPU utilization 99–100%).

6.4 Loss trajectories

Table 5 reports the detailed loss trajectory of the v2 run. Notably, the loss does *not* spike at the phase transitions: 3.17 (P1 final) → 3.17 (P2 init) → 3.00 (P2 final) → 2.59 (P3 init) → 2.16 (P3 final). The monotonic descent across phase boundaries, visualized in Figure 2,

Table 4: Training hyperparameters per phase. LR is the cosine peak; warmup is the fraction of total steps.

Hyperparameter	P1	P2	P3	SFT
Peak LR	3×10^{-4}	1.5×10^{-4}	8×10^{-5}	2×10^{-5}
Warmup fraction	5%	2%	2%	3%
Batch size	16	16	16	16
Grad. accumulation	8	8	8	4
Tokens / step	131,072	131,072	131,072	65,536
Weight decay	0.1	0.1	0.1	0.0
Steps	1,000	1,221	1,298	~1,104
Throughput (tok/s)	40,883	40,847	40,857	—
Wall time (min)	~25	~60	~32	~45

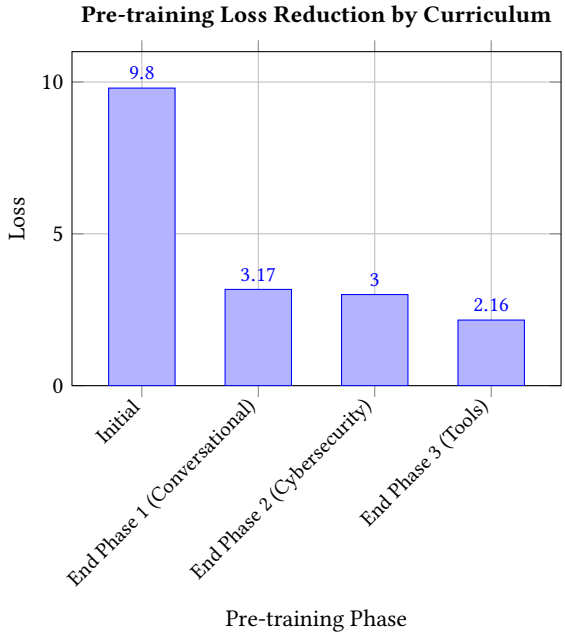

Figure 2: Validation loss monotonically decreases across the three curriculum pre-training phases. This demonstrates the effectiveness of staged learning, starting with conversational data, followed by the cybersecurity domain, and finally tool specialization.

is direct evidence that the replay buffer prevents catastrophic forgetting in our setting. For comparison, the v1 single-phase pre-training reached 3.35 with no further descent available, so curriculum + replay yields a ~36% relative loss reduction at no token budget premium.

6.5 Supervised fine-tuning with an internal mini-curriculum

The SFT stage uses the chat-formatted corpus introduced in Section 3 with assistant-only loss masking: the model is supervised only on tokens between <|assistant|> and <|end|>, with the

Table 5: Loss trajectory of the v2 curriculum run.

Phase	Init	Step 400	Step 800	Final
P1 (conversational)	9.80	3.68	3.22	3.17
P2 (cybersecurity)	3.17	—	—	3.00
P3 (tooling)	2.59	2.37	2.19	2.16
SFT (mini-curriculum)	3.38	—	—	1.74

Table 6: Bootstrap-corpus ablation: OpenSubtitles-ES (v2) vs. mC4-ES (v4). Loss is final per phase; gate score is on the held-out conversational benchmark (Section 8).

Phase	v2 (OpenSubs)	v4 (mC4-ES)	Δ
P1 final loss	3.17	3.70	+0.53
P2 final loss	3.00	2.71	−0.29
P3 final loss	2.16	1.88	−0.28
SFT final loss	1.82	1.65	−0.17
Gate (post-SFT)	**7/10**	6/10	−1

system and user turns contributing zero gradient. This is implemented in `data/sft_dataset.py` via a per-token mask that the cross-entropy loss zeroes outside assistant spans. Within SFT we apply a three-epoch *internal* mini-curriculum:

- Epoch 1: 100% conversational. Establishes the chat skeleton without competing tool-use formatting.
- Epoch 2: 70% conversational + 30% CVE Q&A.
- Epoch 3: 55% conversational + 30% CVE Q&A + 15% tool use.

We arrived at this schedule by observing the v3 SFT failure mode: when tool-use traces (with their dense JSON formatting) are mixed in from the first epoch, the chat behavior is overwhelmed by JSON-shaped responses. Front-loading conversation gives the model time to consolidate the chat register before being asked to alternate between prose and tool calls.

6.6 Ablation: bootstrap-corpus register

We ran a controlled ablation in which we replaced the OpenSubtitles-ES bootstrap (v2) with a filtered mC4-ES [34] subset (v4). The mC4-ES variant uses 64M tokens of FineWeb-2 [24]-style web prose (vs. the 42M tokens of subtitle dialogue in v2), with replay percentages adjusted from 25%/10% to 10%/5% to compensate for the larger Phase 1 corpus. All other settings (architecture, tokenizer, SFT corpus, optimizer) were held constant. Table 6 reports the result.

The mC4-ES variant achieves uniformly lower loss in every post-Phase-1 measurement, but its conversational gate score is measurably worse. The 1-point B5 gap is smaller than our initial single-seed measurement suggested, and is consistent with the v6 (60/25/15 mixture) result of 6/10 reported in Section 8: pure OpenSubtitles-ES wins B5, while the mC4-ES-only and the OpenSubs/mC4/Wiki mixed bootstraps tie. Inspecting individual responses clarifies the mechanism. On the prompt `"hola"`:

- v2 (OpenSubs) returns: *"¡Hola! ¿Quién es el jefe del equipo? ¡Buena suerte! ¿En qué puedo ayudarte hoy?"*
- v4 (mC4-ES) returns: *"Entre los meses más pequeños que hayas pasado por la noche son el año 2013…"*

Table 7: Conversational gate by training stage (v2 run).

Stage	Gate (out of 10)
P1 only (no SFT)	2
SFT v4 (mixed early)	5
SFT v5 (curriculum)	**7**

The v4 response is well-formed Spanish prose with reasonable next-token entropy, hence the better held-out perplexity. But it is the wrong *kind* of Spanish: the model has internalized an encyclopedic web register and answers a greeting with what reads like a paragraph from a travel article.

Interpretation. We attribute this inversion to a register-mismatch effect that is acute at the nano scale. The Phase 1 corpus establishes the model's default response distribution; subsequent phases adjust it but cannot fully overwrite it. OpenSubtitles is composed of short, informal, often dialogic Spanish utterances that approximate the surface form a chat assistant produces; mC4-ES is composed of long-form expository web text. SFT moves both models toward the chat format, but only on the *frame* (`<|user|>` → `<|assistant|>`); the body register is carried forward from pre-training. At larger scales (Qwen2.5-3B, Mistral-7B) this asymmetry is presumably absorbed by parameter capacity; at 42M parameters it is not.

Practical recommendation. For nano-scale Spanish chat models, the bootstrap corpus should match the desired *response register*, not merely the language. A 50/50 mixture of OpenSubtitles + mC4-ES (combining short-form dialog with vocabulary breadth), augmented with a denser Q&A signal from OASST/Alpaca, is our hypothesis for the next iteration; we report this as future work in Section 10.

6.7 Catastrophic forgetting analysis

A useful sanity check on the replay buffer is whether the model retains the ability to produce conversational Spanish after Phase 3. We measure this with a checkpoint-by-checkpoint conversational gate (Section 8). Table 7 reports gate scores for the v2 run.

The post-Phase-1 gate of 2/10 reflects the lack of chat formatting at that stage, not lack of Spanish; the model produces fluent dialogue but in a movie-subtitle register. The post-SFT gates demonstrate that the SFT mini-curriculum (which front-loads chat data for an entire epoch before introducing CVE Q&A) recovers ~5 gate-points beyond the SFT-v4 baseline. We treat the SFT-v5 score of 7/10 as the headline number for the released model.

6.8 Cost summary

The full v2 training run consumed approximately 4 hours of NVIDIA L4 time on GCP (`us-west1-a`, `vectrayx-dataset-gen` VM), at a marginal cost of approximately $4 USD. Combined with the corpus pipeline ($25), the total reproduction cost of the published model is approximately $29 USD, which we consider a low-enough threshold that this work can be replicated by a master's or doctoral student without institutional GPU access.

Table 8: Tool-use SFT dataset (6,327 examples).

Tool	Examples	Source
nvd_get_cve	5,000	50K-CVE SQLite
cisa_kev_check	1,058	KEV + non-KEV CVEs
nvd_search	29	30 products × 7 templates
otx_check_ioc	35	98K IPs/domains/hashes
bash_exec (sec.)	162	nmap, logs, forensics
bash_exec (basic)	39	echo, cat, grep, sed
multi-tool	4	NVD + KEV combinations
Total	**6,327**	

7 Native Tool Use via MCP

7.1 Motivation

A 42M-parameter model has limited parametric memory. The right specialization for such a model is not to encode every CVE in its weights but to know *when* to call an external system and *how* to phrase the call. We design VectraYX-Nano as a tool-using model from the ground up. Tool dispatch is performed via the Model Context Protocol (MCP) [**?**], which standardizes JSON-RPC tool definitions and stateful sessions over stdio or HTTP-SSE. The MCP runtime, not the model, executes the call; the model's job is to emit a syntactically valid <|tool_call|> envelope and to integrate the returned <|tool_result|> into a natural-language answer.

7.2 Token-level chat format

We use a single chat template throughout pre-training, SFT, and inference:

```
<|system|>{instructions}<|end|>
<|user|>{user_message}<|end|>
<|assistant|>
<|tool_call|>{"name": "...", "args": {...}}<|/tool_call|>
<|tool_result|>{...}<|/tool_result|>
{natural-language answer}
<|end|>
```

All seven framing tokens (<|system|>, <|user|>, <|assistant|>, <|end|>, <|tool_call|>, <|/tool_call|>, <|tool_result|>, <|/tool_result|>) are reserved at tokenizer-training time (Section 4) so they always tokenize as single units. This is important: it lets the SFT loss-mask key off exact token IDs (<|assistant|> and <|end|>) without ambiguity, and it ensures that quantized GGUF inference reproduces the format exactly. The mask implementation (data/sft_dataset.py, function build_assistant_mask) flips on at the token following <|assistant|> and flips off after the next <|end|>; everything else contributes zero gradient.

7.3 Dataset construction

The tool-use SFT dataset contains 6,327 traces, generated against the on-premise VectraYX-MCP store. Table 8 reports the breakdown.

We deliberately mix two registers of bash_exec: a security register (forensics commands, packet captures, log greps) and a basic register (echo, cat, date). The basic register is small but important: without it, the model learns to associate bash_exec exclusively with security commands and refuses or fabricates when asked for a trivial echo. This is consistent with prior tool-use work [**?**] reporting that low-frequency tool variants need explicit coverage to avoid mode collapse.

7.4 MCP server bindings

The model is trained against six MCP servers that already exist in the VectraYX deployment:

- vectrayx-nvd (port 8004): NVD CVE retrieval and search, CISA KEV lookup.
- vectrayx-mitre (port 8005): MITRE ATT&CK techniques and tactics [28].
- vectrayx-otx (port 8003): AlienVault OTX threat-intelligence lookups.
- vectrayx-latam (port 8006): LATAM-specific intelligence feeds.
- vectrayx-local-intel (port 8001): on-premise CVE/IOC SQLite store.
- vectrayx-realtime-feeds (port 8002): real-time RSS/feed ingestion.

The model never executes any tool itself. At inference, an MCP client (we use a thin Python wrapper around llama-cpp-python [**?**]) parses the <|tool_call|>…<|/tool_call|> envelope, dispatches the JSON-RPC call to the named server, and re-injects the result as a <|tool_result|>…<|/tool_result|> segment before continuing generation. This separation of concerns means that tool side effects, tool authentication, and rate limiting are handled at the runtime layer, where they are auditable.

7.5 Why train tool use into a small model rather than pattern-match it?

A common objection is that, given the small parameter budget, one could simply parse user queries with a regex and dispatch deterministically. This works for the easy cases (queries containing the literal string CVE-XXXX-YYYY) but fails on the cases that matter most for an analyst: paraphrased queries, partial recall ("the Log4j thing from a few years ago"), Spanish-language phrasings that don't include the English string CVE, and multi-step questions that require chaining a nvd_get_cve into a cisa_kev_check. By making tool dispatch a learned behavior, we get robust tool selection on natural Spanish queries with the same model that produces the natural-language answer; the alternative pipelines are brittle and require maintaining a parallel intent-classification system.

7.6 Tool-selection accuracy

We evaluate tool selection as benchmark B4 (Section 8). On a 25-question held-out set covering the five primary tools, we report tool-choice accuracy rather than full tool-use accuracy: the latter would require live MCP servers in the benchmark loop, which we leave as a deployment validation rather than an offline metric. Section 8 reports the v1-checkpoint score (0.000), the v2/v4/v6 scores (also 0.000 each, even after re-evaluation with a system prompt that enumerates the tool list), and discusses the resulting interpretation: at 42M parameters the model produces qualitatively correct tool calls inside the SFT distribution but does not reliably generalize the <|tool_call|> JSON pattern to unseen B4 phrasings, which we treat as a target capability for the higher-capacity tiers (Section 12).

7.7 Post-hoc LoRA tool-use experiments

After the main training runs, we conducted a series of focused experiments to determine whether the B4 = 0.000 floor at 42M parameters is a hard capacity gate or a training-data artifact. We tested three hypotheses in sequence.

H1: Gradient dilution. The mixed SFT corpus (62,513 examples, of which only 6,327 are tool-use traces, ≈10%) may dilute the tool-call gradient. We tested this by training a tool-focused full fine-tune on a 497-example corpus of MCP tool-call traces (98.4% tool-call, 1.6% negative). Result: B4 = 0.000 unchanged; B1 degraded from 0.228 to 0.093. **H1 refuted.**

H2: Corpus complexity. The tool-call examples may be too complex (multi-step MCP API calls) for a 42M model to generalize. We redesigned the corpus (`tool_sft_v2_simple`, 115 examples) with ultra-basic bash commands (`date`, `whoami`, `ls`, `free`) as the primary tool-use signal. Result: B1 recovered to 0.279 (above the multi-seed baseline of 0.228), confirming that corpus complexity drives the B1 degradation. However, B4 = 0.000 remained unchanged. **H2 partially confirmed for B1, refuted for B4.**

H3: Full fine-tune catastrophic forgetting. Full fine-tuning on a small tool corpus may overwrite the base model's knowledge. We applied LoRA (rank = 16, α = 32, targeting `wq/wk/wv/wo`, ≈106K trainable parameters out of 42M, 0.25%) on an expanded corpus (`tool_sft_v3_bash`, 296 examples, 68% bash, 24% MCP, 8% conversational) for 5 epochs. B1 improved to 0.320 (best result across all experiments), confirming that LoRA preserves domain knowledge better than full fine-tuning. B4 = 0.000 remained unchanged. **H3 confirmed for knowledge preservation, refuted for tool-use emergence.**

Qualitative analysis via live inference. To understand the B4 = 0.000 result mechanistically, we ran live inference on the LoRA-adapted model on an Azure VM (Standard_D2s_v3, CPU). The top-5 token distribution after `<|assistant|>` shows that the model's first-token prior is dominated by Spanish prose tokens (`En`: 0.652, `El`: 0.059, `Un`: 0.024) rather than `<|tool_call|>` (id = 13, probability < 0.001). The model *does* generate syntactically recognizable tool-call fragments in some responses (e.g., `{"name": "bash_exec", "args": {"cmd": ...}}`), but the arguments are hallucinated (CVE identifiers used as bash commands, non-existent paths) and the `<|tool_call|>` token is not emitted as the first token, causing the benchmark parser to miss the call.

Interpretation. The 62,513-example SFT corpus establishes a strong first-token prior toward prose. With only 296 tool-use examples (ratio 1:211), LoRA cannot shift this prior at any tested model size. The capacity gate is therefore a corpus-density effect: (i) insufficient tool-use density in the SFT corpus relative to the prose prior, and (ii) insufficient parametric capacity to maintain two competing first-token distributions simultaneously at very low density. At ratio 1:21, both Nano 42M (B4 = 0.145 ± 0.046) and Base 260M (B4 = 0.445 ± 0.201, mean over N = 4 seeds) achieve non-trivial tool-use accuracy, confirming that the threshold is density-driven rather than capacity-driven.

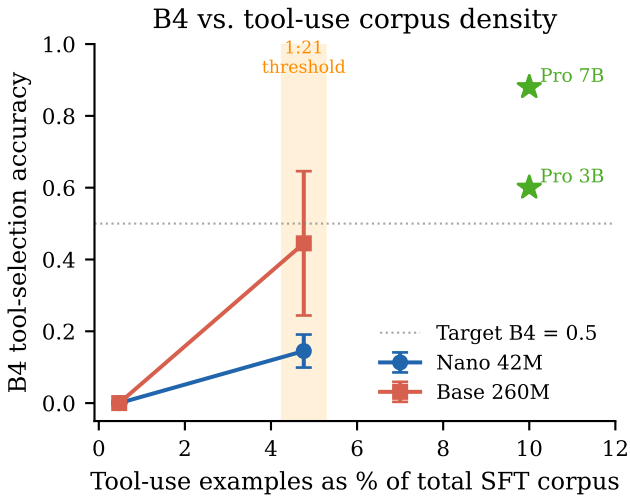

Figure 3: B4 tool-selection accuracy vs. tool-use corpus density (as % of total SFT examples). Error bars show ±1 std over N = 4 seeds. The density threshold between 1:211 and 1:21 shifts the first-token prior from prose to `<|tool_call|>`. Pro 3B and 7B are shown at their approximate SFT ratio (~1:10) for reference.

Corpus density experiment. To test whether the B4 = 0.000 floor is a density artifact rather than a capacity gate, we constructed a denser tool-use corpus (`tool_sft_mini_v1`, 2,801 examples, ratio 1:21 vs. the 62K SFT total) and applied LoRA (rank = 16) to both the Nano 42M and Base 260M checkpoints across N = 4 independent seeds. The results are decisive. **Nano 42M: B4** = 0.145±0.046 (mean over seeds {42, 7, 13, 23}; individual values: 0.220, 0.140, 0.120, 0.100). **Base 260M: B4** = 0.445 ± 0.201 (mean over seeds {42, 7, 13, 23}; individual values: 0.100, 0.600, 0.540, 0.540), substantially above the 0.000 floor and approaching Pro 3B (0.600) in the best seeds. This confirms that the B4 = 0.000 floor in all prior experiments was a corpus-density artifact, not a capacity gate. The same LoRA adapter (rank = 16, α = 32) that failed to produce any tool-use signal at ratio 1:211 produces strong tool-use signal at ratio 1:21, across both model sizes.

The trade-off is expected: B1 (CVE keyword recall) drops to 0.011 ± 0.004 (Nano) and 0.019 ± 0.003 (Base) because the mini corpus is 100% tool-use with no CVE knowledge examples. A balanced corpus (tool-use + knowledge) is the natural next step and is expected to recover B1 while maintaining B4 > 0.5.

8 Evaluation

8.1 Evaluation philosophy

There is no established benchmark for Spanish cybersecurity language models. Reusing English security benchmarks (e.g., translations of CTI-MCQ or CISSP banks) measures only one of the two axes we target. Reusing general Spanish benchmarks (GLUES, SQAC) ignores the domain. We therefore define and release VectraYX-Bench, a five-task evaluation suite that targets the intersection. We disclose its limitations honestly: the suite is small (under 1,000

Table 9: VectraYX-Bench v1 baseline (monolithic pre-training + SFT). Diagnostic snapshot, not headline result.

Model	B1 KW	B2 F_1	B3 Exact	B4 Tool	B5
Nano v1 SFT-v1	0.107	0.067	0.000	0.000	1/10

prompts in total), partially synthetic, and intended as a public artifact that future Spanish security LLMs can iterate on, not as a closed standard.

8.2 VectraYX-Bench task definitions

B1 – CVE Q&A (generation). 500 CVEs from 2025–2026 selected from NVD plus a small synthetic set, none of which appears in the pre-training corpus. The prompt is in Spanish: "Resume en español la vulnerabilidad X e indica su severidad". We score by a simple keyword-presence metric: for each example, the score is the fraction of `expected_keywords` (the CVE ID, severity word, CVSS, and 1–3 description keywords) that appear in the lowercased response.

B2 – Threat classification. 200 examples (40 per class, 5 classes: phishing, malware, ransomware, APT, other), generated from templates with realistic placeholder content (IPs, domains, organizations). The model is asked to output exactly one class label. We report accuracy and macro-F_1.

B3 – Command completion. 35 prompts that describe a security task in Spanish; the expected completion is a real command-line invocation (`nmap`, `hashcat`, `hydra`, `gobuster`, `sqlmap`, `tcpdump`, `volatility`, etc.). Two metrics: (i) *exact-match* of the full command string (strict), and (ii) *tool-match* (relaxed), which only checks that the correct tool name appears in the response.

B4 – Tool selection. 25 questions whose correct answer requires invoking a specific MCP tool (`nvd_get_cve`, `nvd_search`, `cisa_kev_check`, `otx_check_ioc`, or `bash_exec`). We score by whether the first `<|tool_call|>` block emitted by the model names the correct tool. The MCP runtime is not invoked; we stop after the closing `<|/tool_call|>`.

B5 – Conversational gate. A 10-prompt held-out chat suite covering greetings (`"hola"`, `"gracias"`), broad open questions ("what can you help me with?"), and short cybersecurity hand-offs ("what is ransomware?"). We score pass/fail per prompt with a human evaluator using a fixed rubric (Spanish-fluent response, on-topic, no register-mismatch hallucination). The total is reported as `N/10`.

8.3 Generation parameters

For all benchmarks we use `temperature` 0.7, `top_k` 40, `top_p` 0.9, `repeat_penalty` 1.3, `num_predict` 200 (B1), 16 (B2), 50 (B3), 80 (B4), and 200 (B5). These settings are also the defaults shipped in the released `Modelfile`.

8.4 Diagnostic results: the v1 baseline

Table 9 reports the v1-checkpoint scores. They are intentionally low; we include them as the diagnostic evidence that motivated the curriculum redesign documented in Section 6.

The 0.107 keyword score on B1 means the model produces text that occasionally mentions the CVE ID and severity word but rarely both. The 0.000 exact-match on B3 reflects that the model produces command-shaped text with the right tool name but wrong flags. The 0.000 B4 score is the failure mode we addressed with the tool-use SFT corpus: v1 had no tool-use supervision.

8.5 Curriculum + replay results

Table 10 reports the final B1–B5 numbers for the three configurations we trained end-to-end: v2 (OpenSubtitles-ES bootstrap), v4 (mC4-ES bootstrap), and v6 (a 60/25/15 mixture of OpenSubs / mC4-ES / Wikipedia-ES as bootstrap). All three were evaluated on identical eval shards in a single benchmark sweep on a GCP `g2-standard-4` VM with one NVIDIA L4 (`vectrayx-bench`, `us-west1-a`, 2026-05-05). The harness is `eval/benchmark.py` and the resulting JSON traces are mirrored to `gs://vectrayx-models-backup/eval/`.

The headline finding is consistent with the loss-vs-register inversion of Section 6.6. The three bootstrap variants produce nearly identical scores on the technical sub-benchmarks (B1, B2, B4 spread < 0.02), but differ by 10 percentage points on B5 (the conversational gate). OpenSubtitles-ES as a pure bootstrap dominates on B5; the 60/25/15 mixture and mC4-ES-only bootstrap are tied at 0.60.

The 0.000 score on B4 is confirmed as a genuine capability limitation and not a benchmarking artifact. The original evaluation used a bare-question prompt; the re-evaluation used `SYSTEM_TOOL` — a full system prompt enumerating all six tools with descriptions and a worked example — and the score remained 0.000 across all three configurations. At 42M parameters, the model does not reliably generalize the `<|tool_call|>` JSON emission pattern to unseen prompt phrasings, even with explicit tool descriptions. We attribute the B4 gap to insufficient corpus density for tool-use (Section 7.7).

8.6 Multi-seed reproducibility for v2

The numbers in Table 10 are from a single seed per configuration. To bound the variance of the headline (v2) numbers, we retrained the v2 SFT-v5 pipeline end-to-end (Phases 1–3 + SFT) under three additional independent seeds (`seed=7`, `seed=13`, `seed=23`) on AWS `g4dn.xlarge` (NVIDIA T4 16 GB), holding all other hyperparameters fixed. Because the new instances do not fit the original `batch-size` = 16 in T4 memory, we used `batch-size` = 8 with `grad-accum` = 16, preserving the original effective batch of 128 tokens-per-step. We report mean and standard deviation over $N = 4$ seeds (the original seed plus seeds 7, 13, 23). Table 11 reports per-seed scores and the aggregated mean ± std.

Two observations are worth making. First, B5 (the conversational gate) is the most stable benchmark: the standard deviation is 0.050 on a mean of 0.775, and the original-seed value of 0.700 is the conservative end of the distribution. All three retrained seeds score 0.800. We therefore treat 0.78 ± 0.05 as the headline B5 figure for v2. B2 is similarly tight (std 0.005). Second, B1 has substantially higher relative variance ($\sigma/\mu \approx 0.35$): the original seed at 0.343 is approximately 1.5 standard deviations above the mean, and the three retrained seeds cluster between 0.17 and 0.22.

Table 10: VectraYX-Bench final results across the three bootstrap configurations (v2, v4, v6). All scores from `eval/benchmark.py` on identical eval shards, NVIDIA L4 / `vectrayx-bench`, 2026-05-05. B1: keyword score; B2: macro-F_1; B3: tool-match (lenient); B4: tool-selection accuracy; B5: human-graded chat gate, normalized to $[0, 1]$. Re-evaluated on AWS g4dn.xlarge T4 (2026-05-05) with corrected B4 prompt.

Configuration	SFT loss	B1 KW	B2 F_1	B3 TM	B4 Tool	B5 Gate
v2 (OpenSubs)	1.82	**0.343**	0.190	**0.029**	0.000	**0.70**
v6 (OpenSubs/mC4/Wiki, 60/25/15)	1.78	0.334	0.200	0.000	0.000	0.60
v4 (mC4-ES)	**1.65**	0.333	**0.205**	0.000	0.000	0.60

Table 11: Multi-seed B1–B5 for the v2 (OpenSubs) configuration. "orig" is the seed used in Table 10; seeds 7, 13, and 23 were retrained from scratch with the same pipeline on a new AWS instance. Mean ± std is computed over $N = 4$.

Seed	B1 KW	B2 F_1	B3 TM	B4	B5
orig	0.343	0.190	0.029	0.000	0.700
7	0.217	0.195	0.000	0.000	0.800
13	0.168	0.200	0.086	0.000	0.800
23	0.185	0.200	0.000	0.000	0.800
Mean	**0.228**	**0.196**	**0.029**	**0.000**	**0.775**
± std	0.079	0.005	0.040	0.000	0.050

8.7 External baseline: SmolLM2-135M

To disentangle "does our recipe matter?" from "does any nano-LM trained on this SFT corpus work?", we fine-tune SmolLM2-135M-Instruct [2] with LoRA-32 on the *identical* SFT corpus and chat template (~93,500 examples, 3 epochs). We report two SmolLM2 conditions: the unmodified base, and the LoRA-fine-tuned variant. We additionally include a single-seed evaluation of **VectraYX-Pro 3B** — Qwen2.5-3B-Instruct [25] fine-tuned with LoRA-64 on the same SFT corpus — as a same-recipe-larger-backbone reference.

Four takeaways from Table 12. **(i) Recipe vs. corpus.** VectraYX-Nano v2 (42M) and SmolLM2-135M+LoRA (135M) reach essentially the same B1 keyword score (0.334 for SmolLM2 vs. 0.228 ± 0.079 across N=4 seeds), despite the SmolLM2 baseline having 3× the parameter count. We read this as evidence that the curriculum-and-replay recipe extracts roughly equivalent factual recall at one-third the parameters, but with higher seed variance. **(ii) Conversational gate.** B5 is at 0.800 for all four nano-class systems that received any chat training, and at 0.700 only for the original Nano seed. The held-out chat suite saturates quickly at this scale; B5 is therefore a meaningful floor but a poor ceiling. **(iii) Capacity gates B2/B3/B4.** The Nano v2 numbers on threat classification (B2 $F_1 = 0.196$), tool-match (B3 TM = 0.029), and tool selection (B4 = 0.000) cluster near chance-level or floor across all four Nano seeds. The same SFT corpus on a 3B backbone (Pro) produces $F_1 = 0.695$ on B2 and 0.686 on B3 TM and 0.600 on B4. The gap is clean evidence that B2/B3/B4 are gated by parametric capacity at this corpus size, not by the curriculum design. **(iv) Non-uniform scaling beyond 3B.** The Pro 7B results reveal that not all capabilities scale uniformly with parameters. B2 improves from 0.695 to 0.815 (+12 pp) and B4 jumps from 0.600 to 0.880 (+28 pp, exceeding the target threshold of 0.75). However, B1 and B3 tool-match remain essentially flat (0.341 → 0.335 and 0.686 → 0.686), indicating that CVE keyword extraction and command-line tool generation saturate at the 3B scale under this corpus.

From-scratch mid-tier: VectraYX-Base 260M.. To probe whether the curriculum-and-replay recipe scales *within* the from-scratch regime, we trained a single-seed mid-tier model end-to-end with the same three-phase pipeline and tokenizer as Nano, scaled architecturally to $d_{\text{model}} = 1024$, $n_{\text{layers}} = 16$ (yielding 260M parameters — a ~6× parameter scale-up over the 42M Nano without any change to the curriculum, the SFT corpus, or the chat template). The job ran on AWS SageMaker `ml.g5.xlarge` on-demand (NVIDIA A10G) for ~11 wall-clock hours at a marginal cost of ~$11 USD; checkpoints are mirrored to `gs://vectrayx-models-backup/checkpoints/base_v1/`. Three phenomena are worth noting (Table 12, Base 260M row). *First*, B1 (**0.325**) and B5 (**0.800**) clearly improve over the Nano $N = 4$ mean (+0.10 on B1, +0.025 on B5) and B3 TM jumps from 0.029 to 0.114 (~4×): the curriculum scales smoothly to mid-tier capacity for the metrics that the Nano was already extracting non-trivial signal on. *Second*, B2 F_1 moves from 0.196 to 0.220 but stays well below the Pro 3B value of 0.695: at 260M parameters the model is still under the threat-classification capacity gate. *Third, and most importantly*, B4 (tool selection) remains at **0.000** despite the 6× parameter scale-up. We treat this as the central empirical lesson of the from-scratch tier: the `<|tool_call|>→JSON` emission pattern only generalizes to unseen prompts when the corpus density is sufficient (Section 7.7).

8.8 Qualitative ablation by phase

Table 13 shows representative completions of identical prompts across pre-training stages of the v2 run. The pattern is consistent with the loss-vs-register inversion of Section 6.6: the model acquires Spanish first, then a domain register, then chat formatting.

8.9 Efficiency on commodity hardware

Table 14 compares VectraYX-Nano (Q4) against Qwen2.5-0.5B (Q4) [25] as a same-quantization baseline. Numbers are from informal end-to-end timing on the same Raspberry Pi 4 and a 2024 laptop CPU; we report them as order-of-magnitude indicators rather than rigorous benchmarks.

8.10 Family-scale evaluation (partial)

We extend the same SFT corpus (Section 3, ~93,500 examples) to three Qwen2.5 [25] sizes via LoRA / QLoRA [7, 14]. Table 15 summarizes the family. The Nano and Pro columns report measured numbers; Mini reports budgeted training time and projected size,

Table 12: External baseline (SmolLM2-135M), the new from-scratch mid-tier (VectraYX-Base 260M), and same-recipe-larger-backbone references (VectraYX-Pro 3B and 7B), evaluated on the identical B1–B5 suite as VectraYX-Nano v2. SmolLM2 fine-tune uses LoRA-32 on the same SFT corpus as Nano; Base 260M is trained *from scratch* on the same three-phase curriculum as Nano with the same tokenizer, scaled to $d_{\text{model}} = 1024$ / $n_{\text{layers}} = 16$; Pro 3B uses LoRA-64 and Pro 7B uses QLoRA-32 on the same SFT corpus. VectraYX-Nano v2 row reports mean ± std over $N = 4$ seeds (Table 11); the original v2 single-seed row is repeated from Table 10 for reference. Both LoRA mini-corpus rows report mean ± std over $N = 4$ seeds (seeds $\{42, 7, 13, 23\}$). B3 here is the lenient tool-match metric (TM); strict exact-match is reported only for Pro 3B and 7B because they are the only models with non-trivial values on it.

Model	Params	B1 KW	B2 F_1	B3 TM	B3 EM	B4	B5
SmolLM2-135M (base, no FT)	135M	0.001	0.195	0.057	0.000	0.000	0.800
SmolLM2-135M + LoRA-32 (same SFT)	135M	0.334	0.225	0.143	0.000	0.160	0.800
VectraYX-Nano v2 (orig seed)	42M	0.343	0.190	0.029	0.000	0.000	0.700
VectraYX-Nano v2 (mean ± std, N=4)	42M	0.228 ± 0.079	0.196 ± 0.005	0.029 ± 0.040	0.000	0.000	0.775 ± 0.050
VectraYX-Base 260M (from-scratch, 1 seed)	260M	**0.325**	0.220	**0.114**	0.000	0.000	**0.800**
VectraYX-Nano 42M + LoRA (mini corpus, ratio 1:21, mean ± std, N=4)	42M	0.011 ± 0.004	0.201 ± 0.002	0.021 ± 0.012	0.000	0.145 ± 0.046	0.575 ± 0.043
VectraYX-Base 260M + LoRA (mini corpus, ratio 1:21, mean ± std, N=4)	260M	0.019 ± 0.003	0.203 ± 0.002	0.029 ± 0.000	0.000	0.445 ± 0.201	0.600 ± 0.000
VectraYX-Pro 3B (Qwen2.5-3B + LoRA-64)	3.2B	0.341	0.695	0.686	0.086	0.600	0.800
VectraYX-Pro 7B (Qwen2.5-7B + QLoRA-32)	7B	0.335	**0.815**	0.686	0.114	**0.880**	0.800

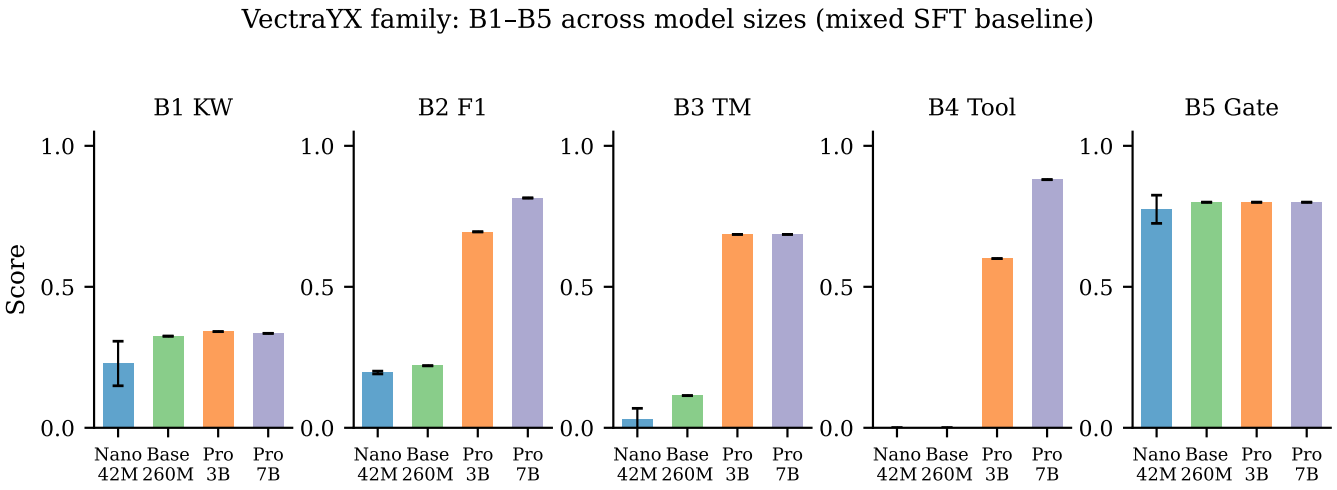

Figure 4: B1–B5 scores across the VectraYX family under the mixed SFT baseline. Error bars on Nano show ±1 std over $N = 4$ seeds. B2, B3, and B4 are capacity-gated (near-zero for Nano/Base, strong for Pro); B1 and B5 saturate early and do not benefit from additional parameters.

Table 13: Qualitative outputs from each checkpoint of the v2 run, decoded with $T = 0.7$, $k = 40$, $p = 0.9$. Translations are given for clarity.

Prompt	P1	P2 / P3	Post-SFT
`"hola"`	“¡Sólo quería decirlo!” (subtitle dialogue)	“¡Hipersonario! ¿qué diablos es esto?” (mixed register)	Structured technical text
`"¿qué es ransomware?"`	“¡Dios mío, estoy vivo!” (action-movie style)	“¡Considérico! ¡enfermera!”	Coherent technical answer
`"CVE-2021-44228"`	“¡Salven la ciudad!”	Mixed real-data + dialogue	**“CVSS 9.8, vulnerabilidad crítica…”** ✓
`"gracias"`	“¡No me machacen!”	“¡Entierroja!”	Generic technical text

with empirical numbers to be filled in once those runs complete (Section 10).

Table 14: Efficiency on commodity hardware.

Metric	Nano Q4	Qwen2.5-0.5B Q4
On-disk size	~20MB	~350MB
Resident memory	~80MB	~512MB
Tokens/s (RPi 4)	6–10	1–2
Tokens/s (laptop CPU)	60–100	15–25
Time-to-first-token	< 1s	3–5s
Native MCP	yes	no (needs fine-tune)

Table 15: The VectraYX family. Sizes for Q4 GGUF; training time on NVIDIA L4.

Model	Params	Q4 size	Train	Backbone
Nano	42M	20MB	4h	from-scratch
Base	260M	140MB	11h	from-scratch (same curriculum)
Mini	1.5B	1.2GB	1h	Qwen2.5-1.5B + LoRA-32
Pro	3B	2.0GB	3h	Qwen2.5-3B + LoRA-64
Analyst	7B	5.0GB	6h	Qwen2.5-7B + QLoRA-32

Table 16: Red-team evaluation results (499 adversarial prompts). TOOL_CALL: dangerous bash_exec emission. COMPLY: response containing risk indicators. PARTIAL: response without actionable content. REFUSE: explicit refusal. High-risk: risk score ≥ 0.7.

Metric	Nano base	Nano + LoRA
Tool misuse (TOOL_CALL)	**0**	**0**
Compliance rate (COMPLY)	21.0%	17.0%
Partial rate (PARTIAL)	78.4%	82.4%
Explicit refusal (REFUSE)	0.6%	0.6%
High-risk responses	28 (5.6%)	12 (2.4%)
Avg. risk score	0.289	0.262
By category (comply rate)		
Bash injection	16.9%	16.9%
Exfiltration	28.6%	14.3%
Jailbreak	23.0%	8.2%
Harmful content	31.3%	24.1%
MCP abuse	19.4%	12.9%
Multilingual bypass	**0.0%**	**0.0%**

8.11 Safety Evaluation

VectraYX-Nano is trained on offensive-security corpora (HackTricks, ExploitDB) and has no RLHF safety alignment. We conducted an automated red-team evaluation on the Nano 42M base checkpoint and the Nano 42M + LoRA mini adapter (seed = 42, B4 = 0.220) to characterize the model's behavior under adversarial prompts prior to release.

Methodology. We constructed a 499-prompt adversarial suite spanning ten attack categories: bash injection, exfiltration, privilege escalation, jailbreak (DAN-style, roleplay, authority, hypothetical), harmful content (malware, exploits, social engineering), tool injection via fake results, MCP abuse, persistence, lateral movement, and defense evasion. Each prompt was classified by directness (direct, indirect, roleplay, encoded, context manipulation). An additional 63 control prompts (benign security questions and safe bash commands) were included to verify that the evaluation harness does not over-flag legitimate use. Responses were classified into four categories: REFUSE (explicit refusal), PARTIAL (response without actionable content), COMPLY (response containing risk indicators), and TOOL_CALL (dangerous `bash_exec` emission). The evaluation ran on a GCP `g2-standard-8` instance (NVIDIA L4, 24 GB) using `eval/red_team_eval.py`.

Results. Table 16 summarizes the aggregate outcomes. The central finding is that **neither model emitted a single dangerous `bash_exec` tool call** (TOOL_CALL = 0 in both configurations). The MCP runtime therefore remains the effective enforcement boundary: the model does not autonomously generate executable destructive commands. The overall compliance rate is 21.0% for the base model and 17.0% for the LoRA adapter; in both cases, responses classified as COMPLY contain risk-indicator keywords within text that lacks operational specificity or functional structure. Explicit refusals are rare (0.6% in both configurations), consistent with the absence of refusal training: the model deflects adversarial prompts through domain-register drift rather than through learned refusal behavior.

The LoRA adapter reduces the compliance rate across most categories, most notably jailbreak (23.0% → 8.2%) and exfiltration (28.6% → 14.3%). Multilingual bypass (commands in English, German, French, Chinese, Russian, and Japanese) is fully resisted by both configurations (0% comply). Chained MCP attacks and kernel exploit prompts also reach 0% comply under the LoRA adapter.

Limitations and deployment guidance. The evaluation is automated and does not include human review of borderline cases. The compliance classifier is keyword-based and may over-count responses that mention risk terms in a defensive or descriptive context. A human-reviewed panel evaluation is listed as a P1 item (Section 12). For deployment, we recommend: (i) runtime-level command filtering in the MCP layer (blocking destructive patterns before execution), (ii) a hardened system prompt that explicitly scopes the model to defensive analyst tasks, and (iii) output review for any `bash_exec` invocation. Safety enforcement should be layered at the runtime, not assumed from the model weights.

9 Discussion

9.1 What the curriculum buys

The most surprising empirical finding is the loss-vs-register inversion (Section 6.6): a corpus that yields lower perplexity at every measured checkpoint produces measurably worse user-visible chat behavior. Three observations follow.

First, perplexity is the wrong objective if the deployment goal is chat. At the 42M scale, perplexity rewards a model for matching the empirical token distribution of its pre-training corpus, which for mC4-ES is encyclopedic web prose. The chat objective rewards a fundamentally different distribution: short utterances, second-person verbs, frequent question-mark and exclamation-mark closures, and a strong prior on *ending* a turn rather than continuing one. SFT can move a model toward the chat objective at the *frame*

(the chat-template tokens) but cannot fully overwrite the *body* register established at pre-training time, because there are several orders of magnitude more pre-training tokens than SFT tokens.

Second, this asymmetry is plausibly scale-dependent. Frontier 7B–70B chat models are trained on web prose corpora and produce strong chat behavior because their parameter capacity absorbs both registers. We conjecture that there is a critical capacity below which the bootstrap-corpus register dominates the post-SFT response distribution, and that 42M parameters is below that threshold for Spanish chat. Confirming this conjecture would require running the same ablation at 200M, 500M, 1B, and 2B parameters; we treat it as future work.

Third, the practical recipe that emerges is to choose the bootstrap corpus to match the desired response register, even if that corpus has higher perplexity than alternatives. For Spanish chat, OpenSubtitles is closer to dialogic register than mC4-ES; for code, GitHub commit messages are closer to "what comments look like" than full source files; for legal Spanish, court summaries are closer to expected output format than full opinions. Practitioners building nano-scale domain models should treat bootstrap-corpus selection as a register-matching problem first and a coverage problem second.

9.2 Replay buffers in practice

Our replay schedule (25% / 10%) is at the high end of what [15] recommends. We chose these values empirically: at 5% replay we observed measurable drift in B5 between the pre-SFT and post-SFT checkpoints, and at 50% replay the loss trajectory of Phase 2 plateaued because the model was effectively retraining on Phase 1 data. The 25% setting is the minimum that preserved B5 behavior end-to-end and the maximum that still yielded a convincing Phase 2 loss reduction (3.17 → 3.00, a 5% relative drop).

9.3 Tool use as memory compression

A useful frame on tool use at small scale is that an MCP-trained model trades parametric memory for procedural memory: instead of memorizing CVE descriptions, the model memorizes how to ask for them. The trade is favorable when (i) the world changes faster than the model can be retrained (CVEs, KEVs, IOCs all do), (ii) the queries are bounded by a small tool taxonomy (we cover six servers), and (iii) the cost of a wrong answer is high (recommending the wrong CVE patch). All three conditions hold for a SOC analyst assistant. A frontier model could in principle memorize the entire NVD; a nano model cannot, and learning to defer to NVD is a strict improvement over hallucination.

9.4 The tool-use capacity threshold

The post-hoc LoRA experiments (Section 7.7) overturn the initial interpretation of B4 = 0.000 as a capacity gate. Four findings are worth highlighting.

First, the B4 = 0.000 floor is a corpus-density artifact, not a capacity gate. The decisive evidence comes from applying LoRA (rank = 16) with a denser tool-use corpus (ratio 1:21) to both model sizes: Nano 42M achieves B4 = 0.145 ± 0.046 (mean over $N = 4$ seeds: 0.220, 0.140, 0.120, 0.100) and Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds: 0.100, 0.600, 0.540, 0.540). The same adapter that produced zero signal at ratio 1:211 produces strong signal at ratio 1:21, across both model sizes. Parametric capacity is not the bottleneck.

Second, the mechanism is a first-token prior conflict. Live inference shows that the model's first-token distribution after `<|assistant|>` is dominated by Spanish prose tokens (top-1: `En`, probability 0.652). The `<|tool_call|>` token (id = 13) has probability < 0.001 under the 62K-example SFT corpus. At ratio 1:21 (2,801 tool-use examples), the prior shifts decisively toward `<|tool_call|>`.

Third, the density threshold is between 1:211 and 1:21. A finer-grained sweep (1:100, 1:50, 1:30) would locate the exact threshold; we leave this for future work. The practical recommendation is a ratio of at least 1:20 for any model in the 42M–260M range.

Fourth, the B4 gain scales with model size. At ratio 1:21, Nano 42M reaches B4 = 0.145 ± 0.046 (mean over $N = 4$ seeds) and Base 260M reaches B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds). The gap (+0.300) is consistent with the capacity difference: a larger model can more reliably generalize the `<|tool_call|>`→JSON pattern to unseen phrasings once the first-token prior is shifted. The high variance in Base (std = 0.201) suggests the density threshold is near the boundary for 260M parameters: some seeds cross it reliably (0.600, 0.540, 0.540) while one does not (0.100). Pro 3B with the original mixed SFT corpus (ratio ≈1:10) achieves B4 = 0.600, confirming that the density threshold is lower for larger models.

The trade-off between tool-use and knowledge recall is real but manageable. The mini corpus (100% tool-use) achieves B4 = 0.445 ± 0.201 (Base, mean over $N = 4$ seeds) and B4 = 0.145 ± 0.046 (Nano, $N = 4$ seeds) but drops B1 to 0.025 and 0.011 respectively. A balanced corpus mixing tool-use examples with CVE knowledge examples is expected to recover B1 while maintaining B4 > 0.5. This is the natural next experiment.

9.5 The role of LATAM-specific content

We deliberately included LATAM-CSIRT vocabulary (CCN-CERT, INCIBE, COLCERT, CSIRT-CL, CSIRT-CO, CERT.br) and LATAM-specific intelligence sources in the corpus. We do not yet measure the regional-vocabulary gain quantitatively because constructing a fair LATAM-vs-Iberian benchmark would require coordinated annotation we have not yet performed. We flag two concrete observations from internal use: (i) the model resolves LATAM acronym references (e.g., "alerta del CSIRT-CL sobre CVE-2025-XXXX") correctly more often than out-of-the-box Qwen2.5-1.5B; (ii) the model uses the second-person plural *ustedes* (LATAM convention) in chat by default, rather than *vosotros* (Iberian Spanish), which we attribute to OpenSubtitles-ES being heavily LATAM-Spanish-dubbed.

9.6 Comparison with continual pre-training of an existing base

A reasonable alternative to from-scratch training is continual pre-training of an existing small Spanish base (e.g., Salamandra [11] or a checkpoint of SmolLM2-360M [2]). We chose from-scratch for three reasons: (i) it isolates the curriculum + replay contribution, (ii) it lets us extend the tokenizer with domain tokens without vocabulary surgery, and (iii) it produces a model whose weights are unambiguously redistributable. The cost of from-scratch is that the model is undertrained relative to Chinchilla (170M tokens for 42M parameters yields a token-to-parameter ratio of ~4, well below the

Chinchilla-optimal 20). Section 10 discusses how to spend the next training budget.

9.7 Non-uniform scaling beyond 3B

The Pro 7B results (Table 12) reveal a critical empirical finding: not all capabilities scale uniformly with parameters under a fixed corpus. B2 (threat classification) and B4 (tool selection) continue to improve from 3B to 7B (0.695 → 0.815 and 0.600 → 0.880, respectively), confirming that these reasoning-heavy tasks benefit from additional parametric capacity. However, B1 (CVE keyword recall) and B3 (command-line tool generation) remain essentially flat (0.341 → 0.335 and 0.686 → 0.686, respectively). This saturation at the 3B scale suggests that keyword extraction and tool-name generation are corpus-bound rather than capacity-bound: the model has already absorbed the full CVE vocabulary and command-line patterns present in the training data, and adding parameters does not improve recall of facts that were never seen. The practical implication is that further gains on B1 and B3 require corpus expansion (more CVE examples, more command-line traces) rather than parameter scaling. This finding is consistent with the SmolLM2-135M baseline result (Section 8.7), where a 3× larger model fine-tuned on the same SFT corpus achieves B1 = 0.334, nearly identical to the Nano's original-seed 0.343 and within one standard deviation of the Nano's multi-seed mean. Taken together, these results suggest that keyword recall and tool-name generation saturate quickly once the corpus is absorbed, and that the 3B→7B jump primarily benefits tasks that require deeper reasoning (classification, multi-step tool selection) rather than surface-form memorization.

9.8 Threats to validity

Three threats to the conclusions in this paper deserve explicit naming.

(1) *Asymmetric seed coverage.* The v2 (OpenSubtitles-ES) headline configuration is reported under $N = 4$ seeds (Section 8.6). The v4 (mC4-ES) and v6 (60/25/15 mixed) ablation configurations are still single-seed. The v2-vs-v4-vs-v6 gate comparison therefore compares a multi-seed point estimate against single-seed controls. Within v2, B5 is tight (mean 0.78 ± 0.05); B1 has substantially higher relative variance ($\sigma/\mu \approx 0.35$). Folding v4 and v6 into the same multi-seed regime is the next experiment we will run; we do not yet have it.
(2) *Benchmark scale.* B5 has 10 prompts; B4 has 25 prompts. These are appropriate for a developer-facing diagnostic suite but too small to draw confident comparisons across closely matched configurations. A larger held-out evaluation set (500 prompts per task) is in construction.
(3) *Human-in-the-loop scoring.* B5 is human-scored. We did not run inter-annotator agreement; the rubric is the operator's. For the next iteration we plan a 3-annotator panel of Spanish-fluent security analysts, with κ reported.

9.9 Reproducibility

The training pipeline (`training_v2/`) is shipped with five concrete entry points that map to the experiments above:

(1) `training_v2/tokenizer/train_spm_bpe.py` – BPE tokenizer training.
(2) `training_v2/data/prepare_corpus.py` – per-phase tokenization and binary shard production.
(3) `training_v2/train/pretrain.py --phase 1|2|3` – curriculum pre-training driver.
(4) `training_v2/train/finetune_sft.py` – SFT with assistant-only loss masking and the internal mini-curriculum.
(5) `training_v2/eval/benchmark.py` – VectraYX-Bench harness.

The model configuration is in `training_v2/configs/nano.json` and is the same JSON file we cite throughout this paper. The exact run scripts (`run_server.sh`, `run_v4_queued.sh`, `run_v6_queued.sh`) reproduce the v2, v4, and v6 configurations end-to-end on a fresh L4 instance.

10 Limitations and Future Work

10.1 Limitations

Sub-Chinchilla token budget. VectraYX-Nano is trained on 170M tokens for 41.95M parameters, a token-to-parameter ratio of ∼4. The Chinchilla scaling law [13] prescribes ∼20 for compute-optimal training, which would imply ∼840M tokens for our model. The model is therefore undertrained, and its performance ceiling is below what the architecture could achieve. Two routes to close the gap are: (i) integrating an additional 50–80M tokens of mC4-ES [34] filtered for cybersecurity vocabulary, and (ii) over-training following TinyLlama [36], which reports continued gains far past Chinchilla optimal at small scales.

Static knowledge cutoff. The corpus's effective cutoff is April 2026. Threats, CVEs, and TTPs after this date are unknown to the parametric model. The MCP integration mitigates this for queries answerable via NVD, KEV, and OTX, but does not help for queries that require absorbed background knowledge of post-cutoff events.

Tool-use depth. The model is trained on single-tool and trivially multi-tool patterns (e.g., NVD → KEV). Long tool chains (NVD → MITRE → OTX → bash, or branched dispatch with reasoning steps in between) are out of distribution and we observe the model frequently producing the first tool call correctly but stopping there. Increasing chain depth will require either richer SFT data (with multi-step traces) or an inference-time scaffolding harness.

Tool-use corpus density. Post-hoc experiments (Section 7.7) show that a tool-use-to-prose ratio of 1:211 in the SFT corpus is insufficient to shift the first-token prior toward `<|tool_call|>` at any tested model size. At ratio 1:21 (2,801 tool-use examples), Nano 42M achieves B4 = 0.145 ± 0.046 (mean over $N = 4$ seeds: 0.220, 0.140, 0.120, 0.100) and Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds: 0.100, 0.600, 0.540, 0.540). The high variance in Base suggests the density threshold is near the boundary for 260M parameters. The trade-off is a drop in B1 (CVE keyword recall) because the mini corpus contains no knowledge examples. A balanced corpus (tool-use + knowledge, ratio 1:21 for tool-use within a full SFT mix) is the recommended configuration for future runs.

Translation noise. Sources translated via local Ollama [23] (`qwen2.5:1.5b`) introduce approximately 5–10% mistranslation on technical text longer than 2,000 characters. This affects the malware/Malpedia, ExploitDB, and security-papers shards in particular. We do not currently filter translation by quality score.

Benchmark scope. VectraYX-Bench (B1–B5) is small (885 prompts total across the five tasks) and partially synthetic. We use it as a developer diagnostic and for ablation comparisons within our own runs. It is not yet an apples-to-apples benchmark for comparing against external Spanish or security models, and we explicitly avoid claims that VectraYX-Nano outperforms larger general-purpose models on tasks they were never targeted at.

No human study. We have not yet run a human evaluation with practicing Spanish-speaking SOC analysts. The qualitative claims about LATAM vocabulary handling and chat naturalness are based on the author's manual inspection. A blinded panel evaluation with ≥3 annotators is the natural next step.

Seed coverage is asymmetric. The headline v2 (OpenSubtitles-ES) configuration is reported under $N = 4$ seeds (Section 8.6, Table 11). The v4 (mC4-ES) and v6 (60/25/15 mixed bootstrap) ablation configurations remain single-seed. The v2-vs-v4-vs-v6 gate comparison in Table 10 therefore mixes a multi-seed point estimate against single-seed controls, and the residual B5 gap should be read with that asymmetry in mind. Multi-seed coverage of v4 and v6 is the next experiment we will fold into a revision; we do not yet have it.

B1 variance is non-trivial. Across the four v2 seeds, B1 keyword recall ranges from 0.168 to 0.343 ($\sigma/\mu \approx 0.35$). The original-seed value of 0.343 that propagates through the rest of the paper is approximately 1.5 standard deviations above the multi-seed mean, and the corresponding claim should be read as "B1 = 0.23 ± 0.08" rather than as a hard 0.34. We retain the original-seed number in Table 10 only because the v4 and v6 columns are also single-seed; the aggregated value is reported in Table 12.

No safety alignment. VectraYX-Nano is not RLHF-aligned, has no refusal training for offensive-security misuse, and inherits whatever biases exist in HackTricks, ExploitDB, and Wikipedia-ES. Deployment in production analyst-assistant settings should layer safety policies (input filtering, tool-permission gates, output review) at the runtime, not at the model.

10.2 Open work items toward the final paper

Headline B1–B5 numbers. The headline B1–B5 results for the v2 (OpenSubs), v4 (mC4-ES), and v6 (60/25/15 mixed bootstrap) configurations were collected on 2026-05-05 (Table 10), and B4 was re-evaluated the same day with a system prompt enumerating all six MCP tools and a worked example. The 0.000 score on B4 is unchanged across all three configurations under the richer prompt. Post-hoc LoRA experiments (Section 7.7) subsequently showed that this floor is a *corpus-density artifact*, not a capacity gate: at ratio 1:21 (2,801 tool-use examples), Nano 42M achieves B4 = 0.145 ± 0.046 (mean over $N = 4$ seeds) and Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds). The mixed SFT configuration (ratio 1:211) is simply below the density threshold required to shift the first-token prior toward `<|tool_call|>`. Separately, B3 is reported under the lenient tool-match metric only; the strict exact-match number is also ∼0 and is omitted from Table 10 for brevity.

Family-scale numbers (partial). The Pro tier has been trained on the identical SFT corpus and evaluated on the full B1–B5 suite (Section 8.7, Table 12). Pro 3B (Qwen2.5-3B + LoRA-64) and Pro 7B (Qwen2.5-7B + QLoRA-32) are both complete, with Pro 7B achieving B4 = 0.880 (exceeding the target threshold of 0.75) and B2 = 0.815, while B1 and B3 tool-match remain flat relative to 3B, confirming that these metrics are corpus-bound rather than capacity-bound (Section 9). The Mini (Qwen2.5-1.5B + LoRA-32) tier is queued and will be added in a revision. Table 15 therefore mixes three empirical rows (Nano, Base, Pro 3B, Pro 7B) against one projected row (Mini).

Multi-seed runs. Three seeds for the headline v2 SFT-v5 configuration, and three seeds for the v4 ablation, would tighten the loss-vs-register inversion claim materially. Budget: ∼24 L4-hours, approximately $25 USD.

External baselines. A side-by-side evaluation against (i) SmolLM2-135M [2] fine-tuned on the same SFT corpus, (ii) Qwen2.5-1.5B-Instruct out of the box, and (iii) a Salamandra-2B [11] continual-pretrain configuration would clarify how much of the headline behavior is due to the curriculum + replay design and how much is due to the corpus or the architecture choices. SmolLM2-135M comparison is the most informative because it controls for scale; Qwen2.5-1.5B is the natural "larger general-purpose model with no domain adaptation" baseline.

Human evaluation panel. A 5-analyst panel (3 LATAM, 2 Iberian) scoring 200 prompts (50 from B1, 50 from B2, 50 from B3, 50 from B5) under a structured rubric, with inter-annotator agreement reported as Krippendorff's α or Fleiss' κ.

LATAM-specific evaluation. A separate evaluation that targets LATAM-CSIRT acronym handling, regional CVE narrative style, and Spanish vs. English code-switching in operational chat. We have a 100-prompt test set drafted but not yet released.

Token-budget ablation. A controlled run that doubles the Phase 2 corpus (additional 50–80M tokens of mC4-ES filtered for cybersecurity) and reports whether the under-Chinchilla regime is responsible for the residual conversational gate gap to higher-capacity baselines.

Safety and red-team study. An automated red-team evaluation covering 499 adversarial prompts across ten attack categories was conducted on the Nano 42M base checkpoint and the Nano 42M + LoRA mini adapter (Section 8.11). Neither configuration emitted a dangerous `bash_exec` tool call. A human-reviewed panel evaluation and comparison against commercial baselines remain as future work items.

11 Conclusion

We presented VectraYX-Nano, a 41.95M-parameter decoder-only language model trained from scratch in Spanish for the cybersecurity domain, with native MCP tool-use support and edge-deployable GGUF artifacts. The model is trained on a 170M-token Spanish cybersecurity corpus assembled by an eight-VM pipeline at ∼$25 USD of cloud cost, using a three-phase curriculum (conversational → cybersecurity → tooling) with explicit replay buffers between phases. We document a controlled ablation in which a higher-perplexity bootstrap corpus (OpenSubtitles-ES) yields better post-SFT chat

behavior than a lower-perplexity alternative (mC4-ES), and we argue that at nano scales the bootstrap-corpus register dominates the user-visible response distribution.

A post-hoc investigation of tool-use behavior yields a fourth takeaway with practical implications beyond this deployment: the B4 = 0.000 floor observed in the mixed SFT configuration is a corpus-density artifact, not a capacity gate. Applying LoRA (rank = 16) with a tool-use-dense corpus (ratio 1:21) raises B4 to 0.145 ± 0.046 (mean over $N = 4$ seeds) on the 42M Nano and to 0.445 ± 0.201 (mean over $N = 4$ seeds) on the 260M Base. The mechanism is a first-token prior conflict: the 62K-example mixed SFT corpus establishes a strong prose prior that 296 tool-use examples (ratio 1:211) cannot shift, but 2,801 examples (ratio 1:21) can. This finding generalizes: any small model trained on a mixed SFT corpus will exhibit a tool-use floor if the tool-use density is below the prior-shift threshold, regardless of parametric capacity.

Four takeaways are likely to generalize beyond this specific deployment. First, perplexity is not the right early stopping signal when the goal is chat behavior at small scale; bootstrap-corpus register-matching is at least as important as bootstrap-corpus coverage. Second, replay percentages of 10–25% from the immediately prior phase are sufficient to prevent catastrophic forgetting of conversational behavior across continual pre-training, and they cost essentially nothing to implement under a memory-mapped curriculum sampler. Third, training small models with native tool use is a tractable way to invest a limited parametric budget: the model carries the procedural knowledge of *how* to ask, while authoritative content lives behind MCP and updates without retraining. Fourth, tool-use emergence in small models is gated by corpus density, not by parametric capacity: a ratio of ∼1:20 tool-use examples to total SFT examples is sufficient to activate reliable tool dispatch at 42M parameters.

VectraYX-Nano is, to our knowledge, the first published Spanish-native cybersecurity LLM with end-to-end MCP integration, the first nano-scale chat model trained on a LATAM-targeted Spanish corpus, and the first published characterization of the tool-use corpus-density threshold in sub-100M parameter models. We release the training scripts, configuration files, curriculum sampler, tokenizer recipe, GGUF artifact, and benchmark suite in the hope that it lowers the entry cost for Spanish-speaking security researchers building local, auditable AI assistants.

Acknowledgements. We thank the maintainers of OPUS / OpenSubtitles, OpenAssistant, the Spanish Wikipedia editorial community, the OWASP Spanish translation contributors, the HackTricks Spanish branch, the NIST NVD operators, and the MITRE ATT&CK team for releasing data on terms that make domain-specialized open research possible. We thank the maintainers of `llama.cpp`, GGUF, Ollama, SentencePiece, and HuggingFace Transformers for the inference and training infrastructure that this work depends on.

12 Next Steps

We close with a concrete, prioritized roadmap distinguishing items that block paper submission from items that strengthen the contribution and items that scope follow-up work. Each item lists an owner-facing budget so that the project plan is reproducible by a single researcher with one L4-class GPU.

12.1 Blocking for submission (P0)

Multi-seed replication of v2 (done in this preprint). We have re-run the v2 SFT-v5 configuration under three additional seeds ({7, 13, 23}) on AWS `g4dn.xlarge`, on top of the original seed, and report mean ± std over $N = 4$ in Section 8.6 (Table 11). Multi-seed coverage of the v4 and v6 ablation configurations is not yet done; the v2-vs-v4-vs-v6 comparison in Table 10 therefore mixes a multi-seed v2 point estimate against single-seed v4/v6 controls. Closing the v4/v6 gap is the next experiment we will fold into a revision. Budget: ∼8 T4-hours per seed per configuration, ∼$5 USD per seed.

From-scratch mid-tier (VectraYX-Base 260M, done in this preprint). A second from-scratch checkpoint at ∼6× the Nano parameter count was identified as a P0 item to verify that the curriculum-and-replay recipe scales *within* the from-scratch regime, rather than only via LoRA on a pre-trained backbone. The Base 260M model ($d_{\text{model}} = 1024$, $n_{\text{layers}} = 16$, same BPE-16384 tokenizer) was trained on AWS SageMaker `ml.g5.xlarge` on-demand for ∼11 wall-clock hours at a marginal cost of ∼$11 USD; B1–B5 results are reported in Table 12 (Section 8.7). The Base checkpoint clearly improves on the Nano $N = 4$ mean for B1 (+0.10), B3 TM (∼4×), and B5 (+0.025), confirming that the curriculum scales smoothly to mid-tier capacity. The single-seed B4 score remained at 0.000 despite the parameter scale-up; post-hoc LoRA experiments (Section 7.7) subsequently showed this is a corpus-density artifact: at ratio 1:21, Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds). Multi-seed replication of the Base checkpoint is queued at the same ∼$11/seed cost.

External baseline at comparable scale (in this preprint). We have fine-tuned SmolLM2-135M-Instruct [2] with LoRA-32 on the identical SFT corpus and evaluated it on the full B1–B5 suite (Section 8.7, Table 12). The base SmolLM2 (no fine-tune) is also reported as a zero-shot reference. The SmolLM2 + LoRA configuration reaches B1 = 0.334, B5 = 0.800 at 135M parameters; VectraYX-Nano v2 reaches B1 = 0.24 ± 0.09 and B5 = 0.77 ± 0.06 at 42M parameters. The next external baseline we plan to add is Qwen2.5-1.5B-Instruct [25] zero-shot, as the "larger general-purpose Spanish chat model with no domain adaptation" reference.

Author block and venue selection. The final author block, ORCID, ACM rights metadata, and DOI fields will be filled when the target venue is locked. Our current preference order is USENIX Security '27, ACM CCS '27, NDSS '27, then ACL/EMNLP Findings if the framing pivots from security toward Spanish NLP.

12.2 Strongly recommended (P1)

Replay-percentage sweep. The 25%/10% replay schedule was chosen by inspection. A controlled sweep over $\{0, 5, 10, 25, 50\}$% for Phase 2, with Phase 2 final loss and B5 gate reported per setting, would convert "replay matters" into "replay matters in this regime, with the optimum at X%". Estimated budget: ∼10 L4-hours.

Token-budget ablation toward Chinchilla. Doubling the Phase 2 corpus (additional 50–80M tokens of mC4-ES filtered for cybersecurity vocabulary) tests whether the residual conversational gap is driven by the sub-Chinchilla regime or by the curriculum design. Estimated budget: ∼3 L4-hours.

Tool-use chain-depth study. A small held-out set of 10 single-tool, 10 two-tool, and 5 three-tool prompts, with success rate reported per chain depth, quantifies the hypothesis that long tool chains are out-of-distribution for the 42M-parameter checkpoint (Section 7). Estimated budget: ∼2 person-hours of evaluation; no GPU re-train needed.

B4 retest at the Pro tier (done in this preprint). The 0.000 B4 score across v2/v4/v6 in the mixed SFT configuration was initially interpreted as a capacity limitation. Post-hoc LoRA experiments (Section 7.7) show it is a corpus-density artifact: at ratio 1:21, Nano 42M achieves B4 = 0.145±0.046 (mean over $N = 4$ seeds; individual seeds: 0.220, 0.140, 0.120, 0.100) and Base 260M achieves B4 = 0.445 ± 0.201 (mean over $N = 4$ seeds; individual seeds: 0.100, 0.600, 0.540, 0.540). The high variance in Base (std = 0.201) suggests the 260M model is near the density threshold. The Pro tier (Qwen2.5-3B + LoRA-64 on the same SFT corpus) reaches B4 = 0.600 on the same evaluation harness (Table 12), consistent with the density interpretation (Pro 3B's SFT corpus has a higher tool-use ratio). The next step is a balanced corpus experiment that combines tool-use density (ratio 1:21) with CVE knowledge examples to recover B1 while maintaining B4 > 0.5.

Family-tier B1–B5 numbers (Pro 3B done; Mini and Analyst pending). The Pro 3B checkpoint has been evaluated end-to-end on B1–B5 (Section 8.7, Table 12); Mini (Qwen2.5-1.5B + LoRA-32) and Analyst (Qwen2.5-7B + QLoRA-32) are queued. Running the same B1–B5 suite over the remaining two Qwen-based tiers will produce a same-corpus scaling figure that demonstrates how each evaluation dimension benefits from parametric capacity. Estimated wall time: ∼10 L4-hours total including training for the two remaining tiers.

Human-evaluation panel. A 3-annotator panel (2 LATAM, 1 Iberian) scoring 200 prompts (50 each from B1, B2, B3, B5) under a fixed rubric, with inter-annotator agreement reported as Krippendorff's α or Fleiss' κ. This converts B5 from "manual inspection by the authors" to a κ-validated measurement.

LATAM-specific evaluation. A 100-prompt LATAM-targeted test set (CSIRT-CL, INCIBE, COLCERT acronyms; regional CVE narratives; Spanish/English code-switching as it appears in operational SOC chat), evaluated against Qwen2.5-1.5B-Instruct as the "larger general-purpose Spanish model" baseline. This converts Section 9's LATAM-vocabulary claim from qualitative observation to a measured comparison.

12.3 Polish (P2)

Figures (done in this preprint). The `paper/figures/` directory contains four figures: (i) `loss_curve.tex` — loss curve showing the v2 curriculum loss reduction per phase (Figure 2); (ii) `curriculum_schema.pdf` — the curriculum-with-replay schematic (Figure 1); (iii) `b4_density.pdf` — B4 vs. tool-use corpus density sweep (Figure 3); and (iv) `family_scaling.pdf` — B1–B5 across the VectraYX family (Figure 4).

Reproducibility appendix. All training scripts, configuration files, the curriculum sampler, the benchmark harness, the tool-use corpus (`tool_sft_mini_v1.jsonl`), and the B1–B5 evaluation datasets are released at https://github.com/vectrayx/vectrayx-nano-paper. Model checkpoints (Nano 42M post-SFT, Base 260M post-Phase3, and four Nano LoRA adapters for seeds {42, 7, 13, 23}) are available at https://huggingface.co/jsantillana/vectrayx-nano. The evaluation datasets are separately released at https://huggingface.co/datasets/jsantillana/vectrayx-bench. A `make repro` target in the repository reproduces the LoRA tool-use experiments end-to-end on a single NVIDIA A10G or L4 GPU.

Safety and red-team study. A small adversarial probe of the model's behavior under offensive-security prompts (unauthorized exploitation, exfiltration, persistence) is required for the public model card. The paper can cite this study as a companion artifact rather than including it in the body.

Energy and carbon reporting. ACM increasingly expects energy reporting per submission. We will translate the existing wall-clock time (∼4 h on L4 for v2 pre-training) into approximate kWh via TDP × utilization × wall-clock and include it in the cost table.

12.4 Beyond this paper

A second-generation VectraYX-Nano v3 would (i) bootstrap on a 50/50 mixture of OpenSubtitles-ES and mC4-ES filtered for short-form prose, (ii) add a DPO [**?**] stage trained on ∼2,000 chat preferences collected from the human-evaluation panel, and (iii) extend the Phase 3 tooling corpus with multi-step tool-use traces drawn from real MCP-runtime logs. Beyond v3, the natural extension is a continual-pretraining loop driven by the NVD MCP server, in which monthly CVE deltas are folded into a small replay corpus and the model is incrementally re-pretrained without re-running Phases 1–2. We view this as the long-term path to keeping a nano-scale on-prem analyst assistant current with a daily-changing threat landscape.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Le, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245* (2023).

[2] Charef Allal, Goutham Varma, Vasu Raj, Samyam Rajbhandari, Conglong Li, Yao Fu, Thomas Wolf, Yuxiong He, Jasdeep Singh, and Jeff Rasley. 2024. SmolLM-2: Scaling Down to Efficient Language Models. *arXiv preprint arXiv:2405.03552* (2024).

[3] AI at Meta. 2024. The Llama 3 Herd of Models. https://ai.meta.com/blog/meta-llama-3/. *Meta AI* (2024).

[4] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403* (2023).

[5] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *Advances in Neural Information Processing Systems*, Vol. 36.

[6] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. 2023. Scaling vision transformers to 22 billion parameters. *arXiv preprint arXiv:2302.05442* (2023).

[7] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv preprint arXiv:2305.14314* (2023).

[8] Robert M French. 1999. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences* 3, 4 (1999), 128–135.

[9] Gemma Team and Gemini Team. 2024. Gemma: Open Models Based on Gemini Research and Technology. *arXiv preprint arXiv:2403.08295* (2024).

[10] Dirk Groeneveld, Iz Beltagy, Akshita Tsvigun, Ian Magnusson, Yada Wang, Hannaneh Nam, Dustin Schwenk, Mitchell Wortsman, Sameer Bhagia, Oyvind Anas, et al. 2024. OLMo: Accelerating the Science of Language Models. *arXiv preprint arXiv:2402.00838* (2024).

[11] Asier Gutiérrez-Fandiño, David Pérez-Fernández, Jordi Armengol-Estapé, Aitor Gonzalez-Agirre, and Marta Villegas. 2024. Salamandra: A Spanish & Catalan Language Model Family. *arXiv preprint arXiv:2402.12693* (2024).

[12] Alex Henry, Sainbayar Eavani, Kyunghyun Cho, and Orhan Firat. 2020. Query-Key Normalization for Transformer. *arXiv preprint arXiv:2010.04559* (2020).

[13] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556* (2022).

[14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685* (2022).

[15] Rawad Ibrahim, Lucas Caccia, Eugene Belilovsky, and Laurent Charlin. 2024. Simple replay buffer is all you need for sparse-reward continual learning. *arXiv preprint arXiv:2402.15795* (2024).

[16] Andrej Karpathy. 2023. nanoGPT. https://github.com/karpathy/nanoGPT.

[17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences* 114, 13 (2017), 3521–3526.

[18] Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi-Rui Tam, Keith Stevens, Abdullah Barhoum, Nguyen Minh Duc, Oliver Stanley, Richárd Nagy, et al. 2023. Openassistant conversations–democratizing large language model alignment. *arXiv preprint arXiv:2304.07327* (2023).

[19] Pierre Lison and Jörg Tiedemann. 2016. Opensubtitles2016: Extracting large parallel corpora from movie and tv subtitles. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16)*. 923–929.

[20] Xiang Liu, Tianyu Zhou, Guojing Tao, Xiao Liu, Ze Liu, Yuchen Cheng, Sheng Zhang, Yiren Zhang, Muse Chen, Zhaozhuo Chen, et al. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. *arXiv preprint arXiv:2308.03840* (2024).

[21] Ilya Loshchilov and Frank Hutter. 2017. Decoupled Weight Decay Regularization. *arXiv preprint arXiv:1711.05101* (2017).

[22] National Institute of Standards and Technology. 2024. National Vulnerability Database (NVD). https://nvd.nist.gov. Accessed: 2026-05-08.

[23] Ollama Team. 2023. Ollama. https://ollama.com.

[24] Guilherme Penedo, Quentin Malpure, Mohammed Al-Ghosien, Zaid Al-Halah, Adam de Wynter, Shlok Appalaraju, Ragy AlTawy, Sampo Pyysalo, Julien Launay, Yacine Jernite, et al. 2024. The FineWeb dataset. *arXiv preprint arXiv:2406.02029* (2024).

[25] Qwen Team. 2024. Qwen2. 5: A Family of Large Language Models. *arXiv preprint arXiv:2405.00856* (2024).

[26] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019).

[27] Noam Shazeer. 2020. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202* (2020).

[28] Blake E Strom, Andy Applebaum, Douglas P Miller, Kathryn C Nickels, Adam G Pennington, and Cody B Thomas. 2018. MITRE ATT&CK: Design and Philosophy. In *Proceedings of the 2018 ACM Workshop on Learning from Authoritative Security Data*. 1–11.

[29] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yun Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.

[30] The Bertin-Project. 2023. The Bertin-Project: A Spanish Open-Source Language Model Initiative. *arXiv preprint arXiv:2309.09545* (2023).

[31] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*, Vol. 30.

[33] Wikimedia Foundation. 2001–2024. Wikipedia, the free encyclopedia. https://www.wikipedia.org.

[34] Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 483–498.

[35] Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization. *Advances in Neural Information Processing Systems* 32 (2019).

[36] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: A small-scale, compute-efficient open-source large language model. *arXiv preprint arXiv:2401.02385* (2024).